

Computer Architecture

CS 622 / EE 520

Fall 2026

Lecture 11
Shahid Masud

- Advanced Branch Predictors and Examples
- Dynamic Scheduling
- Tomasulo Algorithm for Dynamic Scheduling
 - Reservation Station
 - Architecture Details
 - Working
 - CDB
 - Fields in Different Status Registers
- Solved Example of Scoreboard Scheduling
- Solved Example of Tomasulo Scheduling

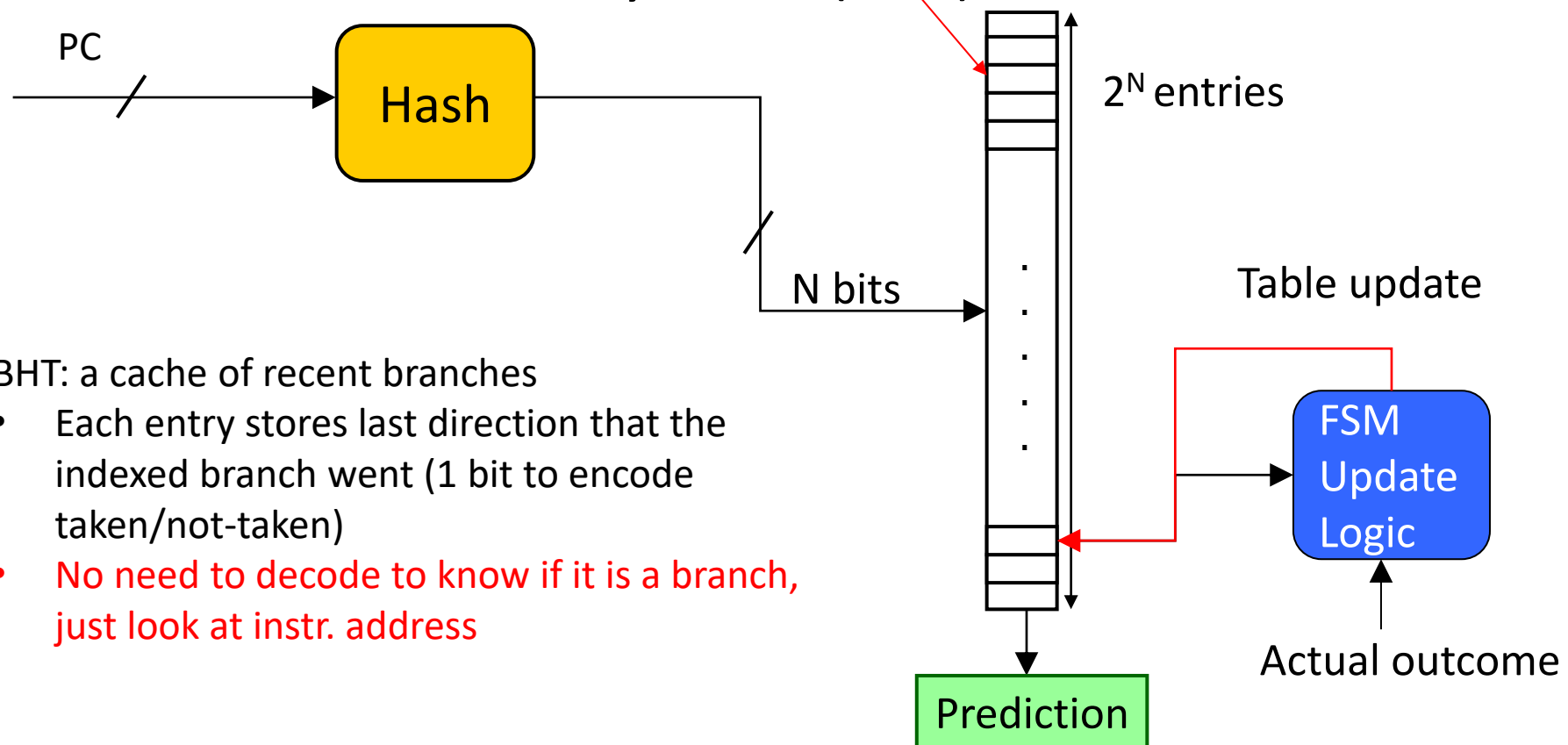
Quiz 3
Next Week

Advanced Branch Predictors

- Static Predictor
 - Taken / No-Taken
- Basic 1-bit predictor
- Basic 2-bit predictor
 - For each branch:
 - Predict taken or not taken
 - If the prediction is wrong two consecutive times, change prediction
- Correlating predictor
 - Multiple 2-bit predictors for each branch
 - One for each possible combination of outcomes of preceding n branches
 - (m,n) predictor: behavior from last m branches to choose from 2^m n -bit predictors
- Tournament predictor
 - Combine correlating predictor with local predictor

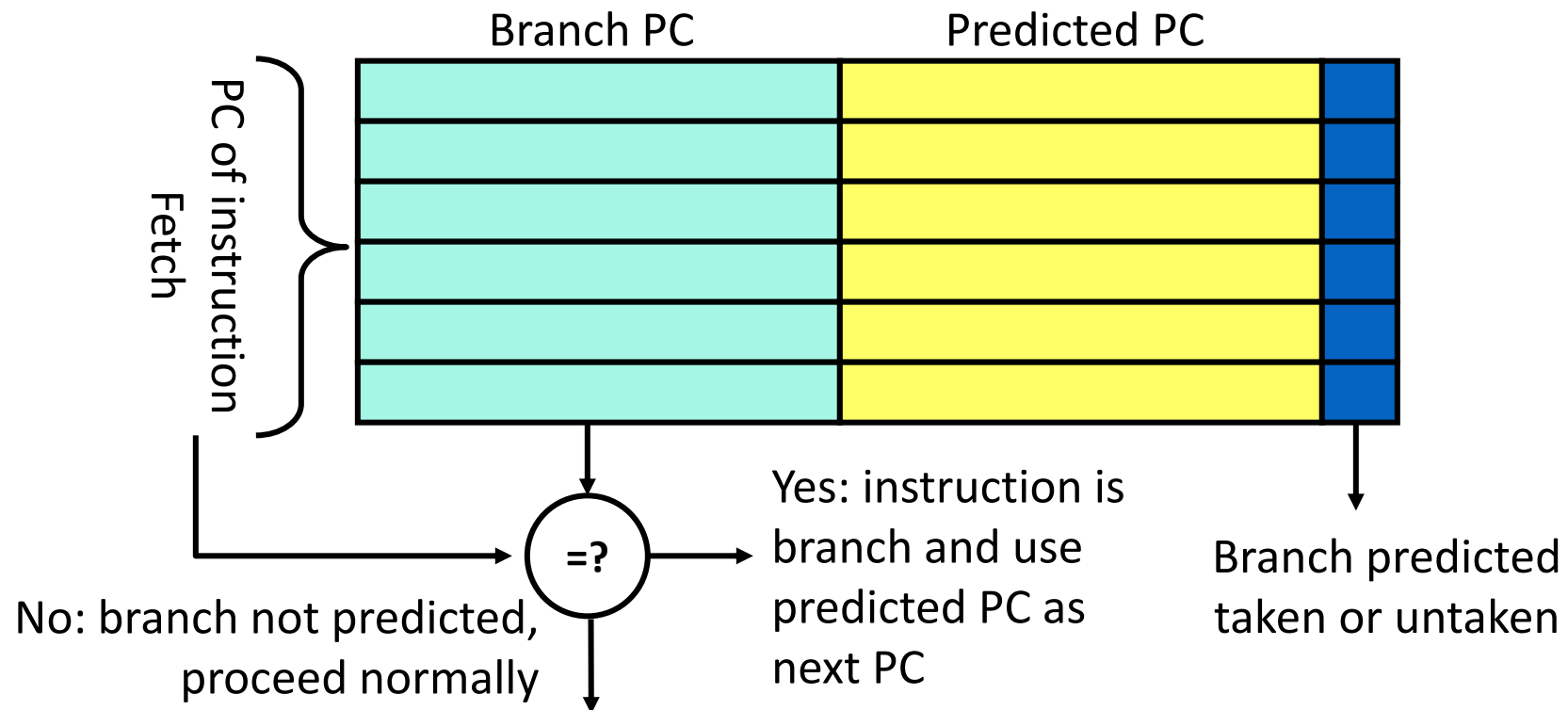
Dynamic Branch Prediction for Direction

- Predict branch based on past history of branch
- One-bit *Branch History Table* (BHT)



Prediction of Branch Targets

- Need target address at same time as prediction
 - *Branch Target Buffer* (BTB): use PC to access I\$ and simultaneously look up BTB to get prediction AND branch address (if taken)



observed dynamic behavior of a single branch in isolation. As the number of instructions in flight has increased with deeper pipelines and more issues per clock, the importance of more accurate branch prediction has grown. In this section, we examine techniques for improving dynamic prediction accuracy. This section makes extensive use of the simple 2-bit predictor covered in Section C.2, and it is critical that the reader understand the operation of that predictor before proceeding.

Correlating Branch Predictors

The 2-bit predictor schemes in Appendix C use only the recent behavior of a single branch to predict the future behavior of that branch. It may be possible to improve the prediction accuracy if we also look at the recent behavior of *other* branches rather than just the branch we are trying to predict. Consider a small code fragment from the eqntott benchmark, a member of early SPEC benchmark suites that displayed particularly bad branch prediction behavior:

Looking at Other Branch Decisions

```

if (aa==2)
    aa=0;
if (bb==2)
    bb=0;
if (aa!=bb) {

```

Here is the RISC-V code that we would typically generate for this code fragment assuming that `aa` and `bb` are assigned to registers `x1` and `x2`:

```

        addi x3,x1,-2
        bnez x3,L1          //branch b1  (aa!=2)
        add  x1,x0,x0       //aa=0
L1:     addi x3,x2,-2
        bnez x3,L2          //branch b2  (bb!=2)
        add  x2,x0,x0       //bb=0
L2:     sub  x3,x1,x2        //x3=aa-bb
        beqz x3,L3          //branch b3  (aa==bb)

```

Let's label these branches `b1`, `b2`, and `b3`. The key observation is that the behavior of branch `b3` is correlated with the behavior of branches `b1` and `b2`. Clearly, if neither branches `b1` nor `b2` are taken (i.e., if the conditions both evaluate to true and `aa` and `bb` are both assigned 0), then `b3` will be taken, because `aa` and `bb` are clearly equal. A predictor that uses the behavior of only a single branch to predict the outcome of that branch can never capture this behavior.

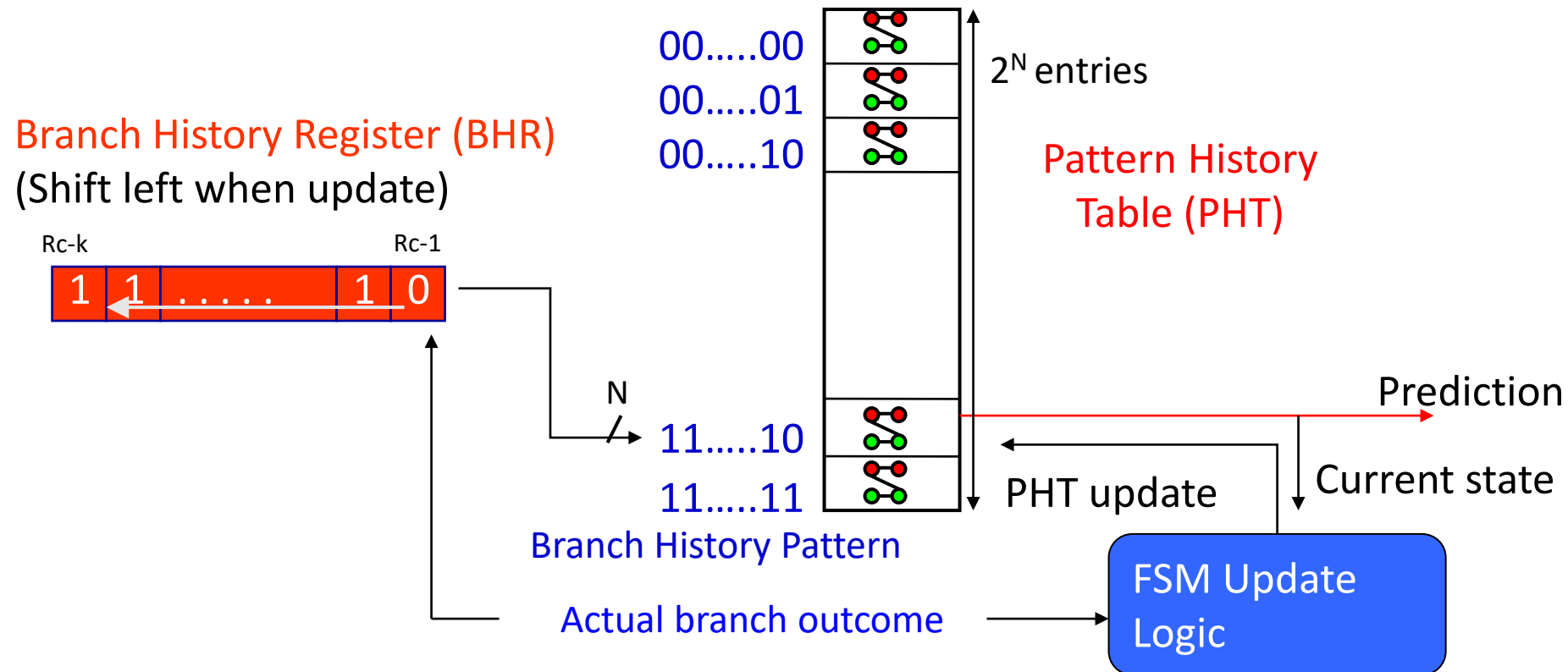
Correlating / 2 Level Branch Predictors



Branch predictors that use the behavior of other branches to make a prediction are called *correlating predictors* or *two-level predictors*. Existing correlating predictors add information about the behavior of the most recent branches to decide how to predict a given branch. For example, a (1,2) predictor uses the behavior of the last branch to choose from among a pair of 2-bit branch predictors in predicting a particular branch. In the general case, an (m , n) predictor uses the behavior of the last m branches to choose from 2^m branch predictors, each of which is an n -bit predictor for a single branch. The attraction of this type of correlating branch predictor is that it can yield higher prediction rates than the 2-bit scheme and requires only a trivial amount of additional hardware.

Two Level Global Branch Prediction

- 1st level: Global Branch History Register (N bits)
 - The direction of last N branches
- 2nd level: Table of saturating counters for each history entry



Example of Correlating Predictor

The simplicity of the hardware comes from a simple observation: the global history of the most recent m branches can be recorded in an m -bit shift register, where each bit records whether the branch was taken or not taken. The branch-prediction buffer can then be indexed using a concatenation of the low-order bits from the branch address with the m -bit global history. For example, in a (2,2) buffer with 64 total entries, the 4 low-order address bits of the branch (word address) and the 2 global bits representing the behavior of the two most recently executed branches form a 6-bit index that can be used to index the 64 counters. By combining the local and global information by concatenation (or a simple hash function), we can index the predictor table with the result and get a prediction as fast as we could for the standard 2-bit predictor, as we will do very shortly.

How much better do the correlating branch predictors work when compared with the standard 2-bit scheme? To compare them fairly, we must compare predictors that use the same number of state bits. The number of bits in an (m,n) predictor is

$$2^m \times n \times \text{Number of prediction entries selected by the branch address}$$

A 2-bit predictor with no global history is simply a (0,2) predictor.

How many bits are in the (0,2) branch predictor with 4K entries? How many entries are in a (2,2) predictor with the same number of bits?

The predictor with 4K entries has

$$2^0 \times 2 \times 4K = 8K \text{ bits}$$

How many branch-selected entries are in a (2,2) predictor that has a total of 8K bits in the prediction buffer? We know that

$$2^2 \times 2 \times \text{Number of prediction entries selected by the branch} = 8K$$

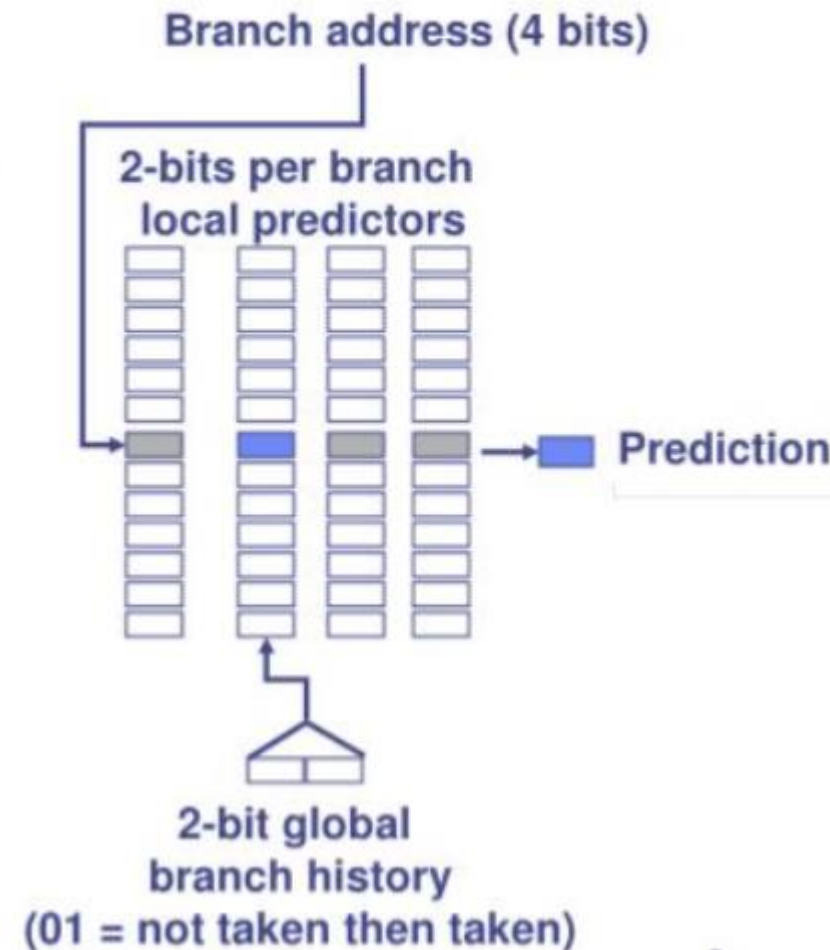
Therefore the number of prediction entries selected by the branch = 1K.

Correlating Branch Predictor

Download

General form: (m, n)
predictor

- m bits for global history, n bits for local history
- Records correlation between m+1 branches
- Simple implementation: global history can be store in a shift register
- Example: (2,2) predictor, 2-bit global, 2-bit local

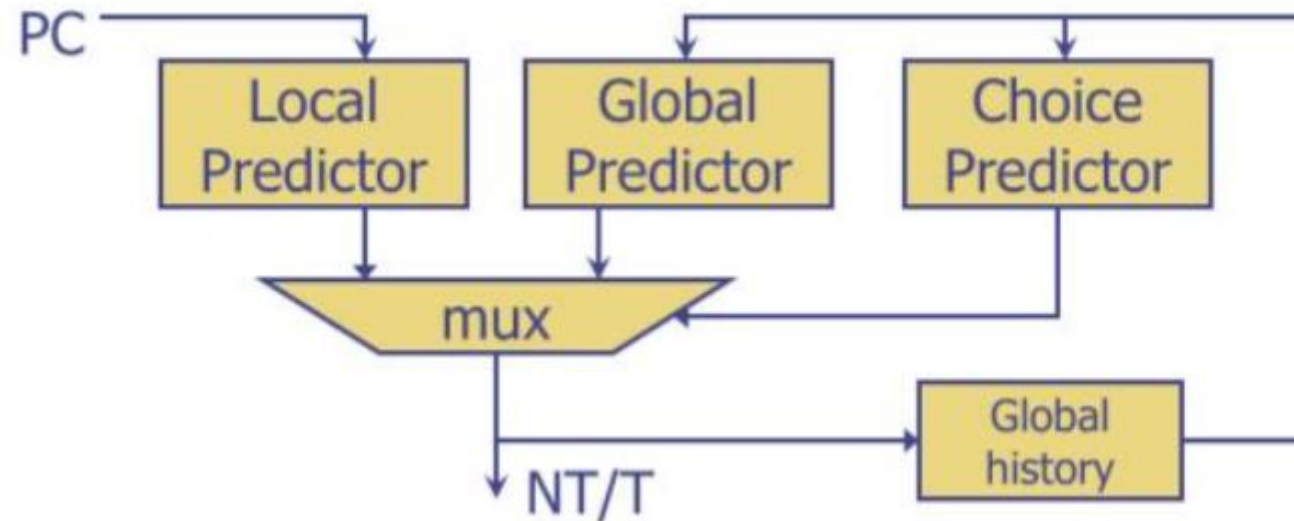


Tournament Predictors: Adaptively Combining Local and Global Predictors

The primary motivation for correlating branch predictors came from the observation that the standard 2-bit predictor, using only local information, failed on some important branches. Adding global history could help remedy this situation. *Tournament predictors* take this insight to the next level, by using multiple predictors, usually a global predictor and a local predictor, and choosing between them

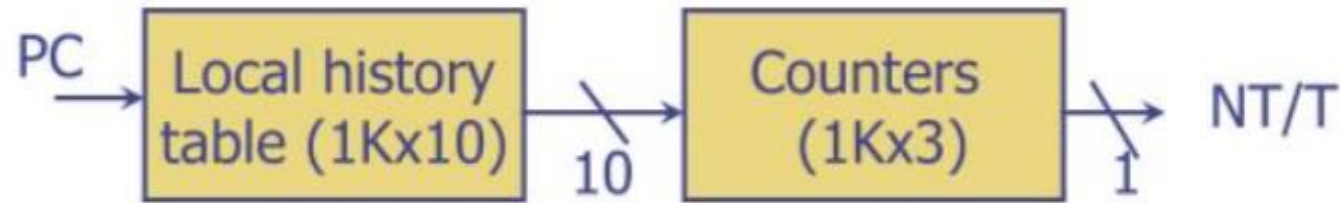
Tournament Branch Predictor

- ◆ Used in Alpha 21264: Track both “local” and global history
- ◆ Intended for mixed types of applications
- ◆ Global history: T/NT history of past k branches, e.g. 0 1 0 1 0 1 (NT T NT T NT T)

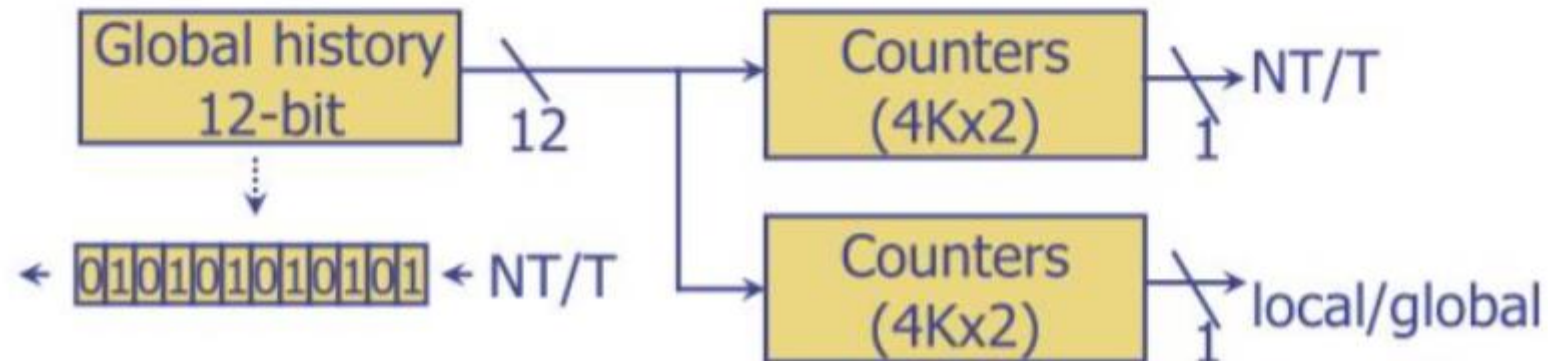


Tournament Branch Predictor

- ◆ Local predictor: use 10-bit local history, shared 3-bit counters



- ◆ Global and choice predictors:



with a selector, as shown in Figure 3.5. A *global predictor* uses the most recent branch history to index the predictor, while a *local predictor* uses the address of the branch as the index. Tournament predictors are another form of hybrid or alloyed predictors.

Tournament predictors can achieve better accuracy at medium sizes (8K–32K bits) and also effectively use very large numbers of prediction bits. Existing tournament predictors use a 2-bit saturating counter per branch to choose among two different predictors based on which predictor (local, global, or even some time-varying mix) was most effective in recent predictions. As in a simple 2-bit predictor, the saturating counter requires two mispredictions before changing the identity of the preferred predictor.

The advantage of a tournament predictor is its ability to select the right predictor for a particular branch, which is particularly crucial for the integer benchmarks.

Gshare 2-bit Predictor

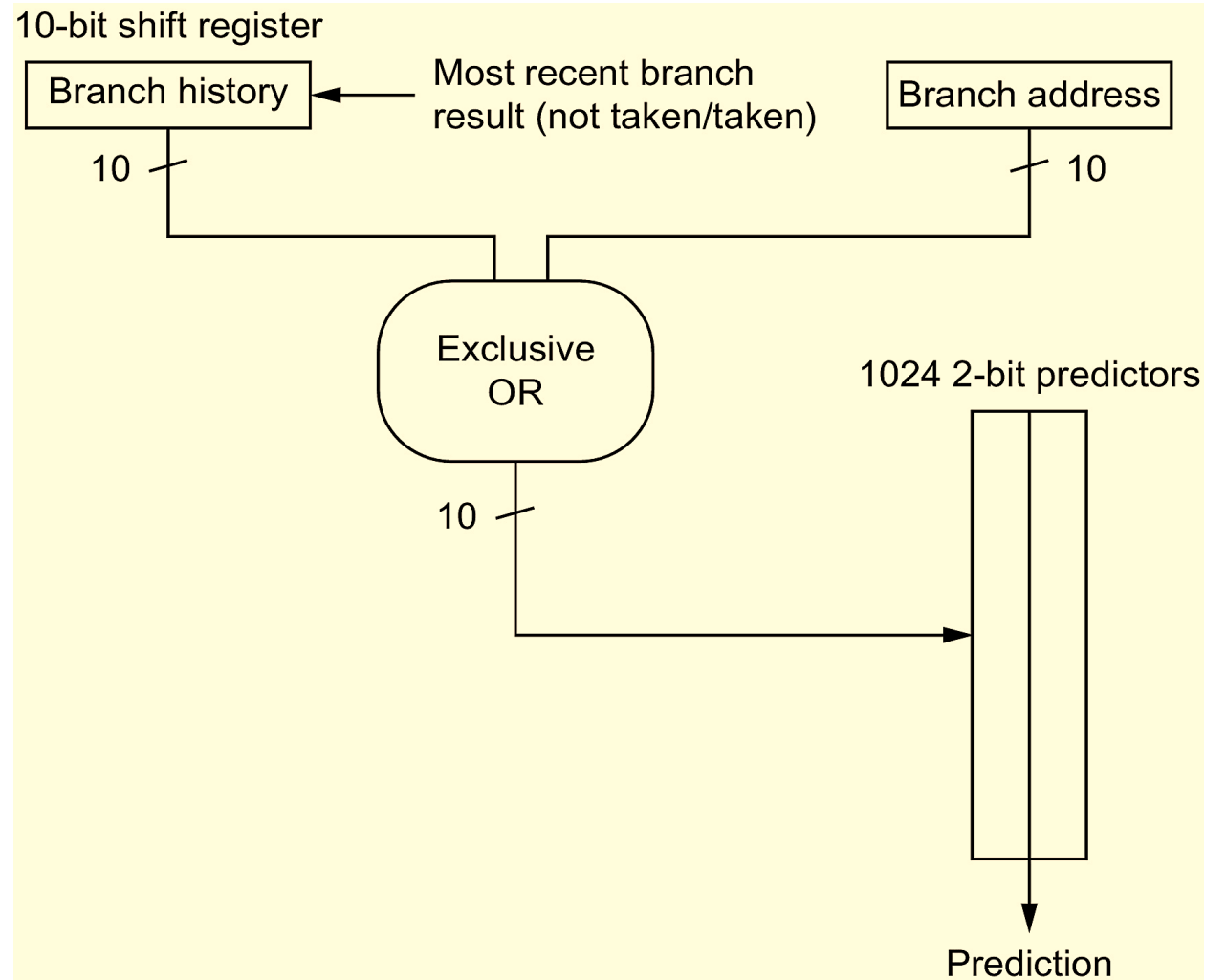


Figure 3.4 A gshare predictor with 1024 entries, each being a standard 2-bit predictor.

Tournament Global Predictor

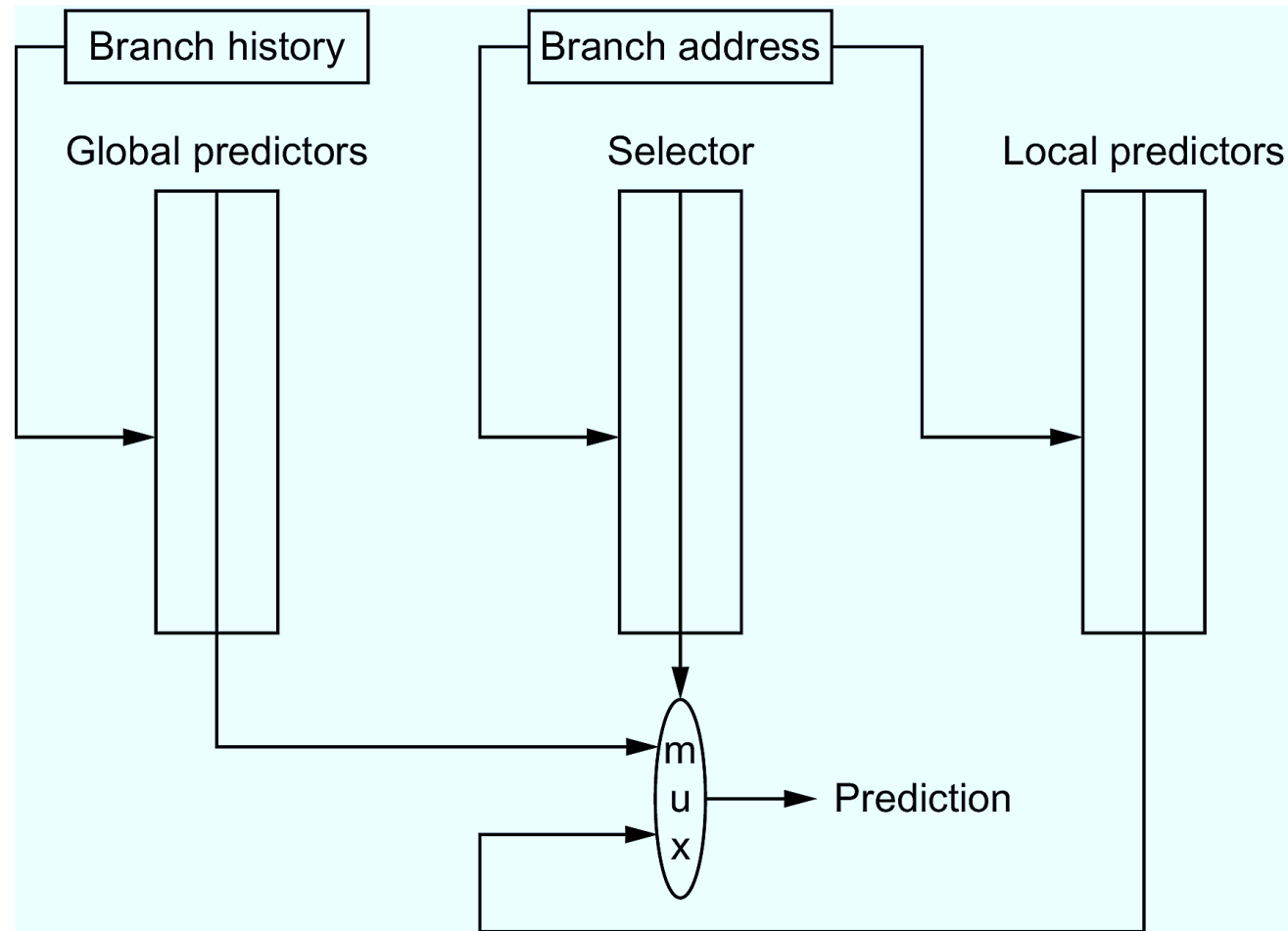
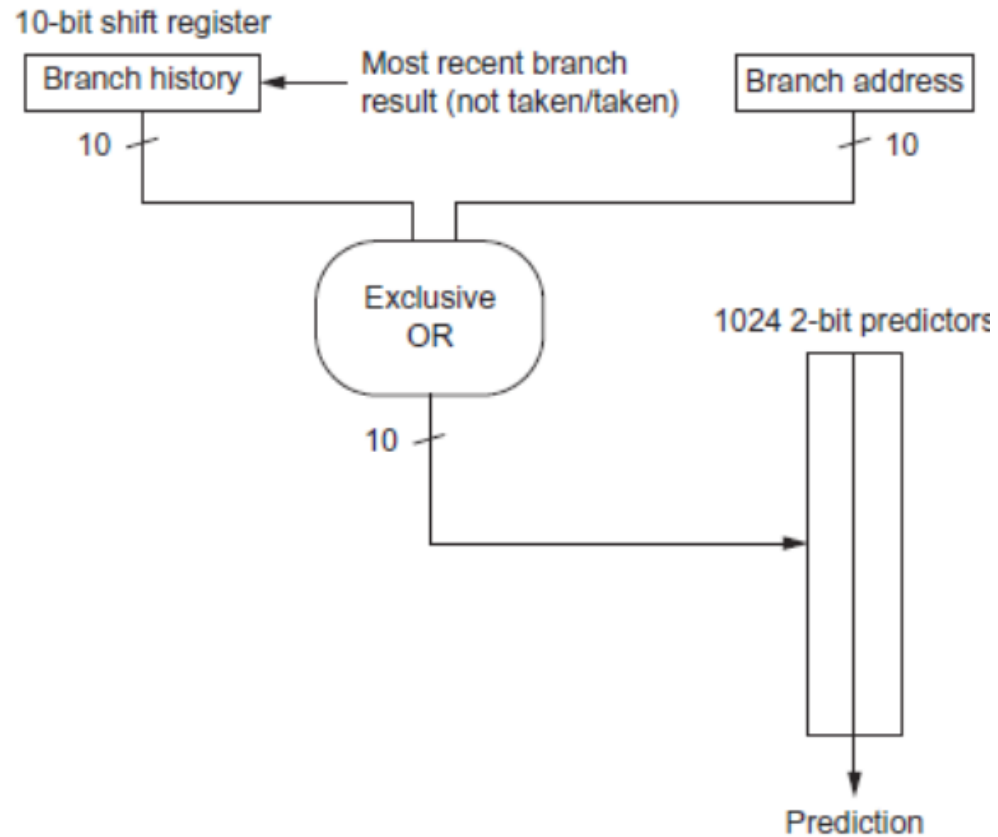
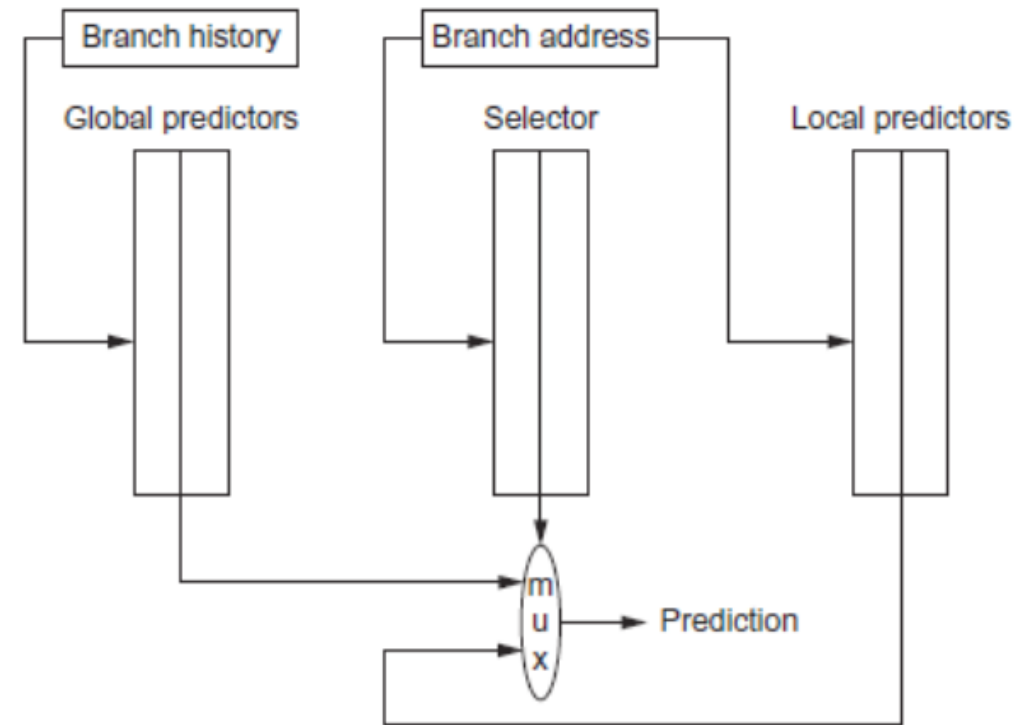


Figure 3.5 A tournament predictor using the branch address to index a set of 2-bit selection counters, which choose between a local and a global predictor. In this case, the index to the selector table is the current branch address. The two tables are also 2-bit predictors that are indexed by the global history and branch address, respectively. The selector acts like a 2-bit predictor, changing the preferred predictor for a branch address when two mispredicts occur in a row. The number of bits of the branch address used to index the selector table and the local predictor table is equal to the length of the global branch history used to index the global prediction table. Note that misprediction is a bit tricky because we need to change both the selector table and either the global or local predictor.

Diagram of Global and Tournament Predictor



Gshare 2-Bit Predictor



Tournament Global Predictor

Tagged Hybrid Predictors

The best performing branch prediction schemes as of 2017 involve combining multiple predictors that track whether a prediction is likely to be associated with the current branch. One important class of predictors is loosely based on an algorithm for statistical compression called PPM (Prediction by Partial Matching). PPM (see Jiménez and Lin, 2001), like a branch prediction algorithm, attempts to predict future behavior based on history. This class of branch predictors, which we call *tagged hybrid predictors* (see Seznec and Michaud, 2006), employs a series of global predictors indexed with different length histories.

Tagged Hybrid Predictor

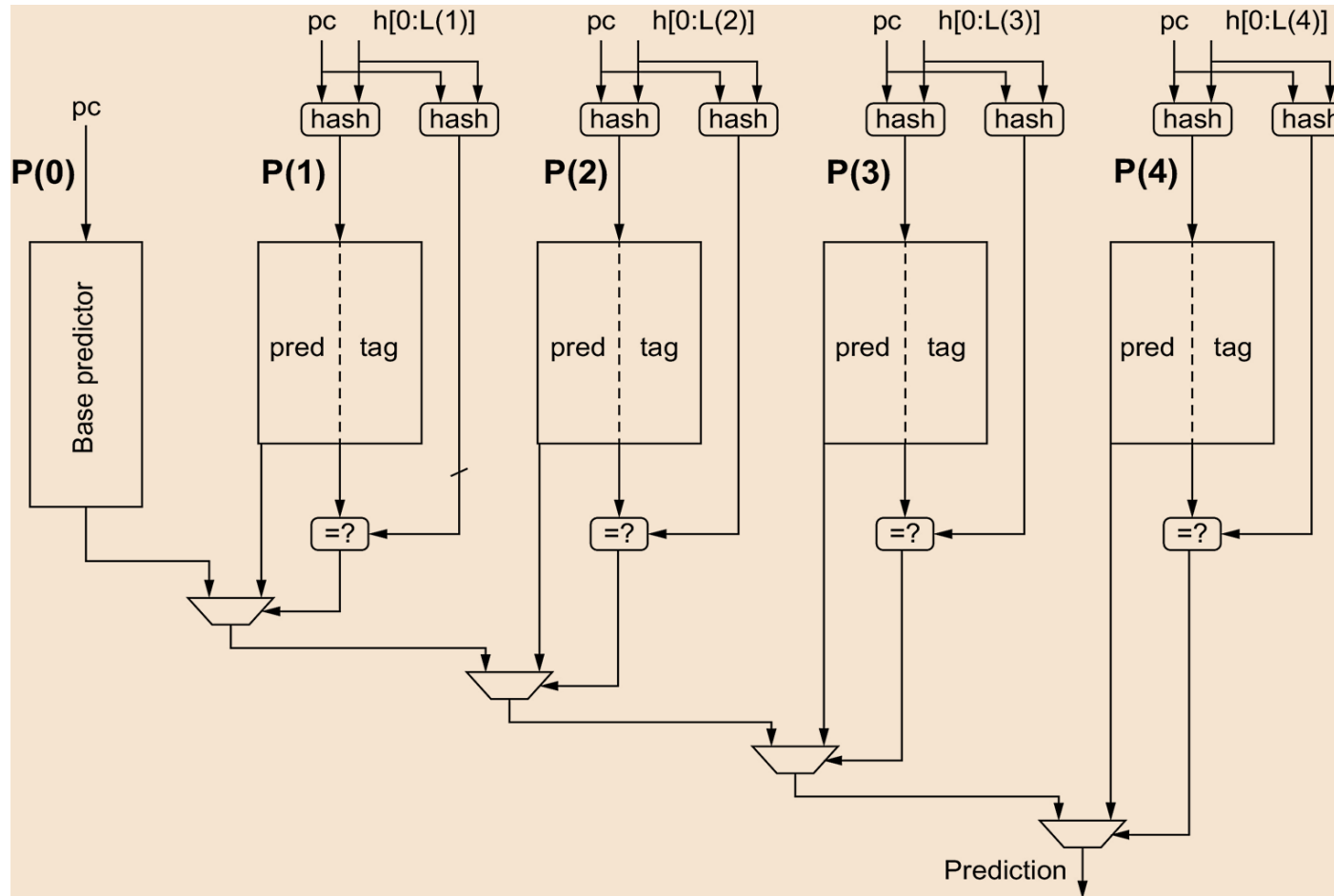


Figure 3.7 A five-component tagged hybrid predictor has five separate prediction tables, indexed by a hash of the branch address and a segment of recent branch history of length 0–4 labeled “h” in this figure. The hash can be as simple as an exclusive-OR, as in gshare. Each predictor is a 2-bit (or possibly 3-bit) predictor. The tags are typically 4–8 bits. The chosen prediction is the one with the longest history where the tags also match.

Example of Core i7 Predictor



The Core i7 920 used a two-level predictor that has a smaller first-level predictor, designed to meet the cycle constraints of predicting a branch every clock cycle, and a larger second-level predictor as a backup. Each predictor combines three different predictors: (1) the simple 2-bit predictor, which is introduced in Appendix C (and used in the preceding tournament predictor); (2) a global history predictor, like those we just saw; and (3) a loop exit predictor. The loop exit predictor uses a counter to predict the exact number of taken branches (which is the number of loop iterations) for a branch that is detected as a loop branch. For each branch, the best prediction is chosen from among the three predictors by tracking the accuracy of each prediction, like a tournament predictor. In addition to this multilevel main predictor, a separate unit predicts target addresses for indirect branches, and a stack to predict return addresses is also used.

A few words about ISCA Conference



<https://www.iscaconf.org/isca2025/program/workshops.php>

https://ericrotenberg.wordpress.ncsu.edu/cbp2025/

NC STATE UNIVERSITY

Electrical and Computer Engineering →

Eric Rotenberg

About Me Teaching Research Publications Talks Students

6th Championship Branch Prediction (CBP2025)

in conjunction with:

ISCA-52 <http://iscaconf.org/isca2025/index.html>

Sponsored by ARM

The organizers of the 6th Championship Branch Prediction (CBP2025) invite you to participate in a competition for the best branch predictor.

Workshop Program

https://freddygabbay.github.io/AGPC-ISCA2025/

The logo for the AGPC 2025 workshop is a square with a green border. Inside, there are horizontal blue and white stripes, and a green triangle on the right side with the text 'AGPC 2025' in white.

AGPC 2025 Workshop - Call for Papers

ARM-based General-Purpose Computing: Software-Hardware Co-Optimization for Performance Acceleration

June 21st 2025 @ Tokyo, Japan

Room: Okuma-large

Held in conjunction with [ISCA 2025](#)

About the Workshop

The International Symposium on Computer Architecture (ISCA) Workshop on "ARM General-Purpose Computing: Software-Hardware Co-Optimization for Performance Acceleration" aims to bring together researchers, engineers, and industry leaders to explore the latest advancements and innovations in leveraging ARM architecture for high-performance general-purpose computing in data center. With the growing importance of ARM-based systems in data centers and beyond, this workshop focuses on the synergy between software and hardware to achieve significant performance gains. We invite contributions that address novel algorithms, new benchmarks regarding performance and power, microarchitectural designs, hardware accelerators, tightly-coupled architectures, and optimization techniques tailored for ARM instruction set architecture. This workshop seeks to foster a

Data Hazards and Dynamic Scheduling

Advantages of Dynamic Schedule



flow and exception behavior. Dynamic scheduling offers several advantages. First, it allows code that was compiled with one pipeline in mind to run efficiently on a different pipeline, eliminating the need to have multiple binaries and recompile for a different microarchitecture. In today's computing environment, where much of the software is from third parties and distributed in binary form, this advantage is significant. Second, it enables handling some cases when dependences are unknown at compile time; for example, they may involve a memory reference or a data-dependent branch, or they may result from a modern programming environment that uses dynamic linking or dispatching. Third, and perhaps most importantly, it allows the processor to tolerate unpredictable delays, such as cache misses, by executing other code while waiting for the miss to resolve. In Section 3.6, we explore hardware speculation, a technique with additional performance advantages, which builds on dynamic scheduling. As we will see, the advantages of dynamic scheduling are gained at the cost of a significant increase in hardware complexity.

Although a dynamically scheduled processor cannot change the data flow, it tries to avoid stalling when dependences are present. In contrast, static pipeline

The Idea behind Dynamic Scheduling



A major limitation of simple pipelining techniques is that they use in-order instruction issue and execution: instructions are issued in program order, and if an instruction is stalled in the pipeline, no later instructions can proceed. Thus, if there is a dependence between two closely spaced instructions in the pipeline, it will lead to a hazard, and a stall will result. If there are multiple functional units, these units could lie idle. If instruction j depends on a long-running instruction i , currently in execution in the pipeline, then all instructions after j must be stalled until i is finished and j can execute. For example, consider this code:

```
fdiv.d  f0,f2,f4
fadd.d  f10,f0,f8
fsub.d  f12,f8,f14
```

The `fsub.d` instruction cannot execute because the dependence of `fadd.d` on `fdiv.d` causes the pipeline to stall; yet, `fsub.d` is not data-dependent on anything in the pipeline. This hazard creates a performance limitation that can be eliminated by not requiring instructions to execute in program order.

Checking Hazards in out of order



To allow us to begin executing the `fsub.d` in the preceding example, we must separate the issue process into two parts: checking for any structural hazards and waiting for the absence of a data hazard. Thus we still use in-order instruction issue (i.e., instructions issued in program order), but we want an instruction to begin execution as soon as its data operands are available. Such a pipeline does *out-of-order execution*, which implies *out-of-order completion*.

Out-of-order execution introduces the possibility of WAR and WAW hazards, which do not exist in the five-stage integer pipeline and its logical extension to an in-order floating-point pipeline. Consider the following RISC-V floating-point code sequence:

```
fdiv.d  f0, f2, f4
fmul.d  f6, f0, f8
fadd.d  f0, f10, f14
```

There is an antidependence between the `fmul.d` and the `fadd.d` (for the register `f0`), and if the pipeline executes the `fadd.d` before the `fmul.d` (which is waiting for the `fdiv.d`), it will violate the antidependence, yielding a WAR hazard. Likewise, to avoid violating output dependences, such as the write of `f0` by `fadd.d` before `fdiv.d` completes, WAW hazards must be handled. As we will see, both these hazards are avoided by the use of register renaming.

Exceptions in out of order



Out-of-order completion also creates major complications in handling exceptions. Dynamic scheduling with out-of-order completion must preserve exception behavior in the sense that *exactly those exceptions that would arise if the program were executed in strict program order actually do arise*. Dynamically scheduled processors preserve exception behavior by delaying the notification of an associated exception until the processor knows that the instruction should be the next one completed.

Although exception behavior must be preserved, dynamically scheduled processors could generate *imprecise exceptions*. An exception is *imprecise* if the processor state when an exception is raised does not look exactly as if the instructions were executed sequentially in strict program order. Imprecise exceptions can occur because of two possibilities:

1. The pipeline may have *already completed* instructions that are *later* in program order than the instruction causing the exception.
2. The pipeline may have *not yet completed* some instructions that are *earlier* in program order than the instruction causing the exception.

Imprecise exceptions make it difficult to restart execution after an exception.

Modifications for out of order Execution

To allow out-of-order execution, we essentially split the ID pipe stage of our simple five-stage pipeline into two stages:

- ID
1. Issue—Decode instructions, check for structural hazards.
 2. Read operands—Wait until no data hazards, then read operands.

An instruction fetch stage precedes the issue stage and may fetch either to an instruction register or into a queue of pending instructions; instructions are then issued from the register or queue. The execution stage follows the read operands stage, just as in the five-stage pipeline. Execution may take multiple cycles, depending on the operation.

We distinguish when an instruction *begins execution* and when it *completes execution*; between the two times, the instruction is *in execution*. Our pipeline allows multiple instructions to be in execution at the same time; without this capability, a major advantage of dynamic scheduling is lost. Having multiple instructions in execution at once requires multiple functional units, pipelined functional units, or both. Because these two capabilities—pipelined functional units and multiple functional units—are essentially equivalent for the purposes of pipeline control, we will assume the processor has multiple functional units.

Dependence types



- RAW (Read After Write) = "true dependence"

mul r0 * r1 -> **r2**

...

add **r2** + r3 -> r4



- WAW (Write After Write) = "output dependence"

mul r0 * r1 -> **r2**

...

add r1 + r3 -> **r2**



- WAR (Write After Read) = "anti-dependence"

mul r0 * **r1** -> r2

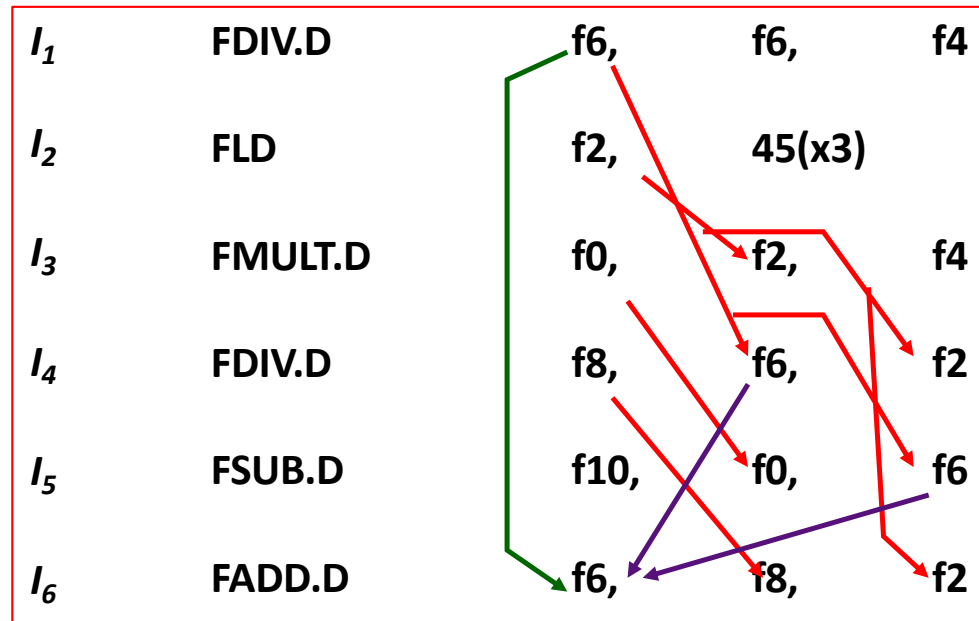
...

add r3 + r4 -> **r1**



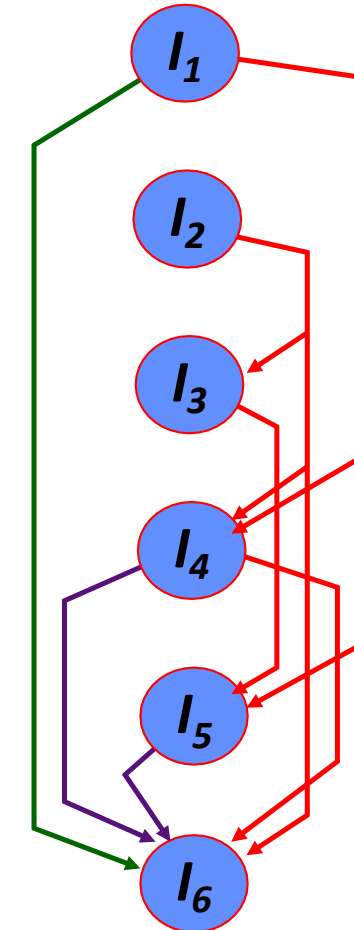
- WAW & WAR are "false", Can be **totally eliminated** by "renaming"

Instruction Scheduling - recap

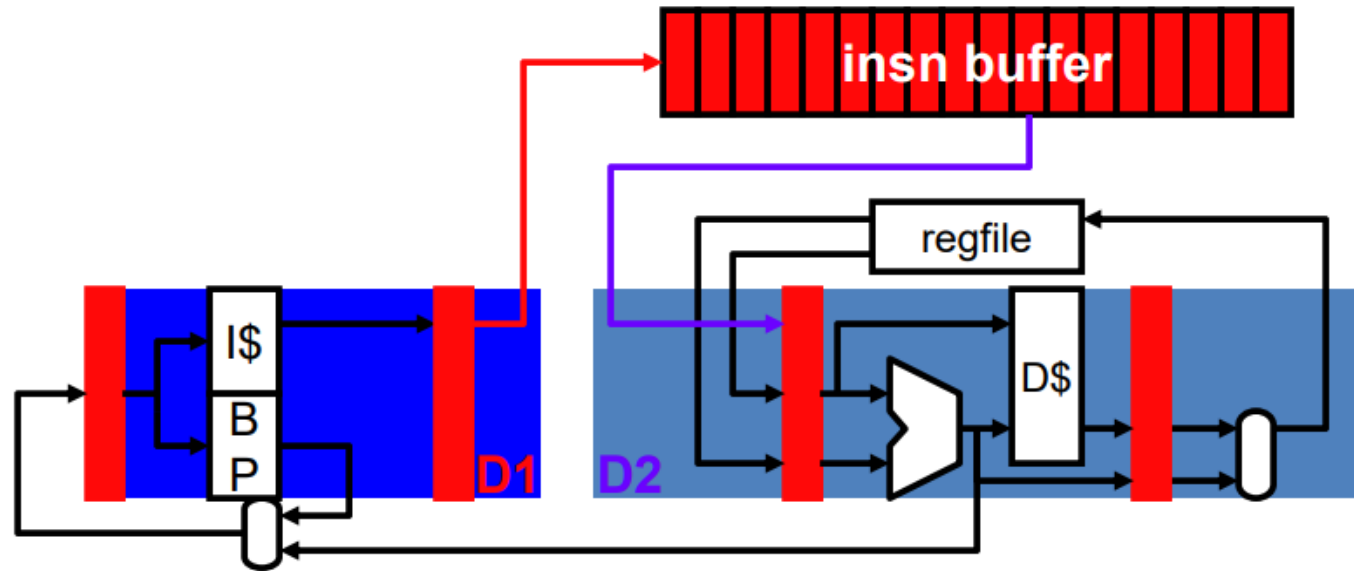


Valid orderings:

in-order	I_1	I_2	I_3	I_4	I_5	I_6
out-of-order	I_2	I_1	I_3	I_4	I_5	I_6
out-of-order	I_1	I_2	I_3	I_5	I_4	I_6



RAW hazards are avoided by executing an instruction only when its operands are available, which is exactly what the simpler scoreboarding approach provides. WAR and WAW hazards, which arise from name dependences, are eliminated by register renaming. *Register renaming eliminates these hazards by renaming all destination registers, including those with a pending read or write for an earlier instruction, so that the out-of-order write does not affect any instructions that depend on an earlier value of an operand.* The compiler could typically implement such renaming, if there were enough registers available in the ISA. The original 360/91 had only four floating-point registers, and Tomasulo's algorithm was created to overcome this shortage. Whereas modern processors have 32–64 floating-point and integer registers, the number of renaming registers available in recent implementations is in the hundreds.



- Trick: **insn buffer** (many names for this buffer)
 - Basically: a bunch of latches for holding insns
 - Implements iteration fusing ... here is your scheduling scope
- Split D into two pieces
 - Accumulate decoded insns in buffer **in-order**
 - Buffer sends insns down rest of pipeline **out-of-order**

- **Purpose of Renaming: removing “Anti-dependencies”**
 - Get rid of WAR and WAW hazards, since these are not “real” dependencies
- **Implicit Renaming: i.e. Tomasulo**
 - Registers changed into values or response tags
 - We call this “implicit” because space in register file may or may not be used by results!
- **Explicit Renaming: more physical registers than needed by ISA.**
 - Rename table: tracks current association between architectural registers and physical registers
 - Uses a translation table to perform compiler-like transformation on the fly

Register Renaming Eliminates WAR, WAW

ards, consider the following example code sequence that includes potential WAR and WAW hazards:

```
fdiv.d  f0,f2,f4
fadd.d  f6,f0,f8
fsd     f6,0(x1)
fsub.d  f8,f10,f14
fmul.d  f6,f10,f8
```

There are two antidependences: between the fadd.d and the fsub.d and between the fsd and the fmul.d. There is also an output dependence between the fadd.d and the fmul.d, leading to three possible hazards: WAR hazards on the use of f8 by fadd.d and its use by the fsub.d, as well as a WAW hazard because the fadd.d may finish later than the fmul.d. There are also three true data dependences: between the fdiv.d and the fadd.d, between the fsub.d and the fmul.d, and between the fadd.d and the fsd.

These three name dependences can all be eliminated by register renaming. For simplicity, assume the existence of two temporary registers, S and T. Using S and T, the sequence can be rewritten without any dependences as

```
fdiv.d  f0,f2,f4
fadd.d  S,f0,f8
fsd     S,0(x1)
fsub.d  T,f10,f14
fmul.d  f6,f10,T
```

In addition, any subsequent uses of f8 must be replaced by the register T. In this example, the renaming process can be done statically by the compiler. Finding any

- RAW (Read After Write) = "true dependence"
mul r0 * r1 -> r2
...
add r2 + r3 -> r4
- WAW (Write After Write) = "output dependence"
mul r0 * r1 -> r2
...
add r1 + r3 -> r2
- WAR (Write After Read) = "anti-dependence"
mul r0 * r1 -> r2
...
add r3 + r4 -> r1
- WAW & WAR are "false", Can be totally eliminated by "renaming"

Register Renaming for Eliminating WAR and WAW Dependencies



- Example:

DIV.D F0,F2,F4
ADD.D F6,F0,F8
S.D F6,0(R1)
SUB.D T2,F10,F14
MUL.D T1,F10,T2



DIV.D F0,F2,F4
ADD.D F6,F0,F8
S.D F6,0(R1)
SUB.D F8,F10,F14
MUL.D F6,F10,F8

- Now only RAW hazards remain, which can be strictly ordered

When to Execute an Instruction



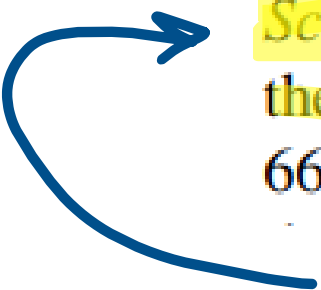
RAW hazards are avoided by executing an instruction only when its operands are available, which is exactly what the simpler scoreboarding approach provides.

WAR and WAW hazards, which arise from name dependences, are eliminated by register renaming. Register renaming eliminates these hazards by renaming all destination registers, including those with a pending read or write for an earlier instruction, so that the out-of-order write does not affect any instructions that depend on an earlier value of an operand. The compiler could typically implement such renaming, if there were enough registers available in the ISA. The original 360/91 had only four floating-point registers, and Tomasulo's algorithm was created to overcome this shortage. Whereas modern processors have 32–64 floating-point and integer registers, the number of renaming registers available in recent implementations is in the hundreds.

- **Dynamic Scheduling of Instructions:**
 - In-order issue
 - Out-of-order execution
 - Out-of-order completion
- **Data Hazard via Register Renaming**
 - Dynamic RAW hazard detection and scheduling in data-flow fashion
 - Register renaming for WRW and WRA hazard (name conflict)
- **Implementations**
 - Scoreboard (CDC 6600 1963)
 - » Centralized register renaming
 - Tomasulo's Approach (IBM 360/91, 1966)
 - » Distributed control and renaming via reservation station, load/store buffer and common data bus (data+source)

In Dynamic Schedule Pipeline, all instructions pass through the Issue Stage “in-order”

other in the second stage (read operands) and thus enter execution out of order. *Scoreboarding* is a technique for allowing instructions to execute out of order when there are sufficient resources and no data dependences; it is named after the CDC 6600 scoreboard, which developed this capability. Here we focus on a more sophis-

A blue curved arrow originates from the left side of the slide and points towards the word 'Scoreboarding' in the paragraph above.

Scoreboard Example is Later in Today's Lecture

Introduce Tomasulo

6600 scoreboard, which developed this capability. Here we focus on a more sophisticated technique, called *Tomasulo's algorithm*. The primary difference is that Tomasulo's algorithm handles antidependences and output dependences by effectively renaming the registers dynamically. Additionally, Tomasulo's algorithm can be extended to handle speculation, a technique to reduce the effect of control dependences by predicting the outcome of a branch, executing instructions at the predicted destination address, and taking corrective actions when the prediction was wrong. While the use of scoreboarding is probably sufficient to support simpler processors, more sophisticated, higher performance processors make use of speculation.

The IBM 360/91 floating-point unit used a sophisticated scheme to allow out-of-order execution. This scheme, invented by Robert Tomasulo, tracks when operands for instructions are available to minimize RAW hazards and introduces register renaming in hardware to minimize WAW and WAR hazards. Although there are many variations of this scheme in recent processors, they all rely on two key principles: dynamically determining when an instruction is ready to execute and renaming registers to avoid unnecessary hazards.

Introducing Tomasulo Dynamic Scheduling

Reservation Stations for Register Renaming



In Tomasulo's scheme, register renaming is provided by reservation stations, which buffer the operands of instructions waiting to issue and are associated with the functional units. The basic idea is that a reservation station fetches and buffers an operand as soon as it is available, eliminating the need to get the operand from a register. In addition, pending instructions designate the reservation station that will provide their input. Finally, when successive writes to a register overlap in execution, only the last one is actually used to update the register. As instructions are issued, the register specifiers for pending operands are renamed to the names of the reservation station, which provides register renaming.

Because there can be more reservation stations than real registers, the technique can even eliminate hazards arising from name dependences that could not be eliminated by a compiler. As we explore the components of Tomasulo's scheme, we will return to the topic of register renaming and see exactly how the renaming occurs and how it eliminates WAR and WAW hazards.

→ The use of reservation stations, rather than a centralized register file, leads to two other important properties. First, hazard detection and execution control are distributed: the information held in the reservation stations at each functional unit determines when an instruction can begin execution at that unit. Second, results are passed directly to functional units from the reservation stations where they are buffered, rather than going through the registers. This bypassing is done with a common result bus that allows all units waiting for an operand to be loaded simultaneously (on the 360/91, this is called the common data bus, or CDB). In pipelines that issue multiple instructions per clock and also have multiple execution units, more than one result bus will be needed.

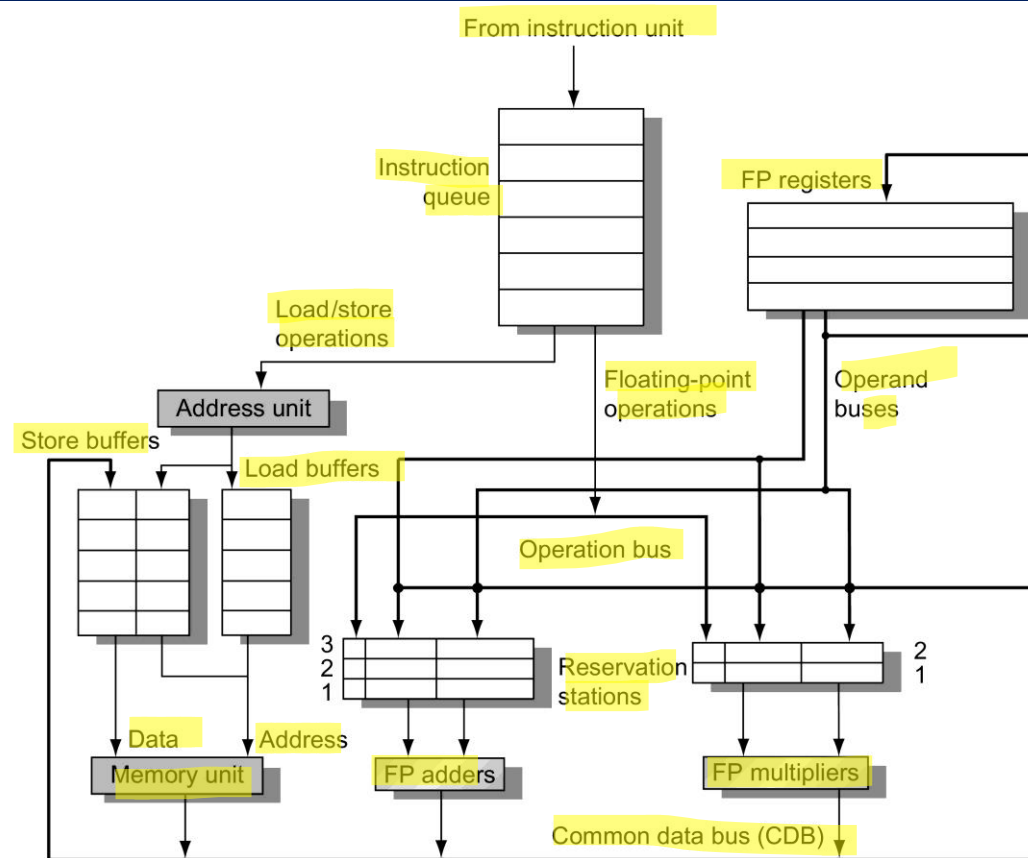


Figure 3.10 The basic structure of a RISC-V floating-point unit using Tomasulo's algorithm. Instructions are sent from the instruction unit into the instruction queue from which they are issued in first-in, first-out (FIFO) order. The reservation stations include the operation and the actual operands, as well as information used for detecting and resolving hazards. Load buffers have three functions: (1) hold the components of the effective address until it is computed, (2) track outstanding loads that are waiting on the memory, and (3) hold the results of completed loads that are waiting for the CDB. Similarly, store buffers have three functions: (1) hold the components of the effective address until it is computed, (2) hold the destination memory addresses of outstanding stores that are waiting for the data value to store, and (3) hold the address and value to store until the memory unit is available. All results from either the FP units or the load unit are put on the CDB, which goes to the FP register file as well as to the reservation stations and store buffers. The FP adders implement addition and subtraction, and the FP multipliers do multiplication and division.

- **Dynamic Scheduling of Instructions:**
 - In-order issue
 - Out-of-order execution
 - Out-of-order completion
- **Data Hazard via Register Renaming**
 - Dynamic RAW hazard detection and scheduling in data-flow fashion
 - Register renaming for WRW and WRA hazard (name conflict)
- **Implementations**
 - Scoreboard (CDC 6600 1963)
 - » Centralized register renaming
 - Tomasulo's Approach (IBM 360/91, 1966)
 - » Distributed control and renaming via reservation station, load/store buffer and common data bus (data+source)

Three Stages of Tomasulo Algorithm



1. Issue—get instruction from FP Op Queue

If reservation station free (no structural hazard),
control issues instr & sends operands (renames registers).

2. Execution—operate on operands (EX)

When both operands ready then execute;
if not ready, watch Common Data Bus for result

3. Write result—finish execution (WB)

Write on Common Data Bus to all awaiting units;
mark reservation station available

- Normal data bus: data + destination (“go to” bus)
- Common data bus: data + source (“come from” bus)
 - 64 bits of data + 4 bits of Functional Unit source address
 - Write if matches expected Functional Unit (produces result)
 - Does the broadcast

Some Details of Three Steps in Tomasulo



1. Issue

1. *Issue*—Get the next instruction from the head of the instruction queue, which is maintained in FIFO order to ensure the maintenance of correct data flow. If there is a matching reservation station that is empty, issue the instruction to the station with the operand values, if they are currently in the registers. If there is not an empty reservation station, then there is a structural hazard, and the instruction issue stalls until a station or buffer is freed. If the operands are not in the registers, keep track of the functional units that will produce the operands. This step renames registers, eliminating WAR and WAW hazards. (This stage is sometimes called *dispatch* in a dynamically scheduled processor.)

2. Execute

2. *Execute*—If one or more of the operands is not yet available, monitor the common data bus while waiting for it to be computed. When an operand becomes available, it is placed into any reservation station awaiting it. When all the operands are available, the operation can be executed at the corresponding functional unit. By delaying instruction execution until the operands are available, RAW hazards are avoided. (Some dynamically scheduled processors call this step “issue,” but we use the name “execute,” which was used in the first dynamically scheduled processor, the CDC 6600.)

3. Write Results

3. *Write result*—When the result is available, write it on the CDB and from there into the registers and into any reservation stations (including store buffers) waiting for this result. Stores are buffered in the store buffer until both the value to be stored and the store address are available; then the result is written as soon as the memory unit is free.

Common
Data
Bus

Tomasulo Example Cycle 3

Instruction status:

Instruction	<i>j</i>	<i>k</i>	Issue	Exec Comp	Write Result	Busy	Address
LD	F6	34+	R2	1	3	Load1	Yes 34+R2
LD	F2	45+	R3	2		Load2	Yes 45+R3
MULTD	F0	F2	F4	3		Load3	No
SUBD	F8	F6	F2				
DIVD	F10	F0	F6				
ADDD	F6	F8	F2				

Reservation Stations:

Time	Name	Busy	Op	S1 <i>V_j</i>	S2 <i>V_k</i>	RS <i>Q_j</i>	RS <i>Q_k</i>
Add1		No					
Add2		No					
Add3		No					
Mult1		Yes	MULTD		R(F4)	Load2	
Mult2		No					

Register result status:

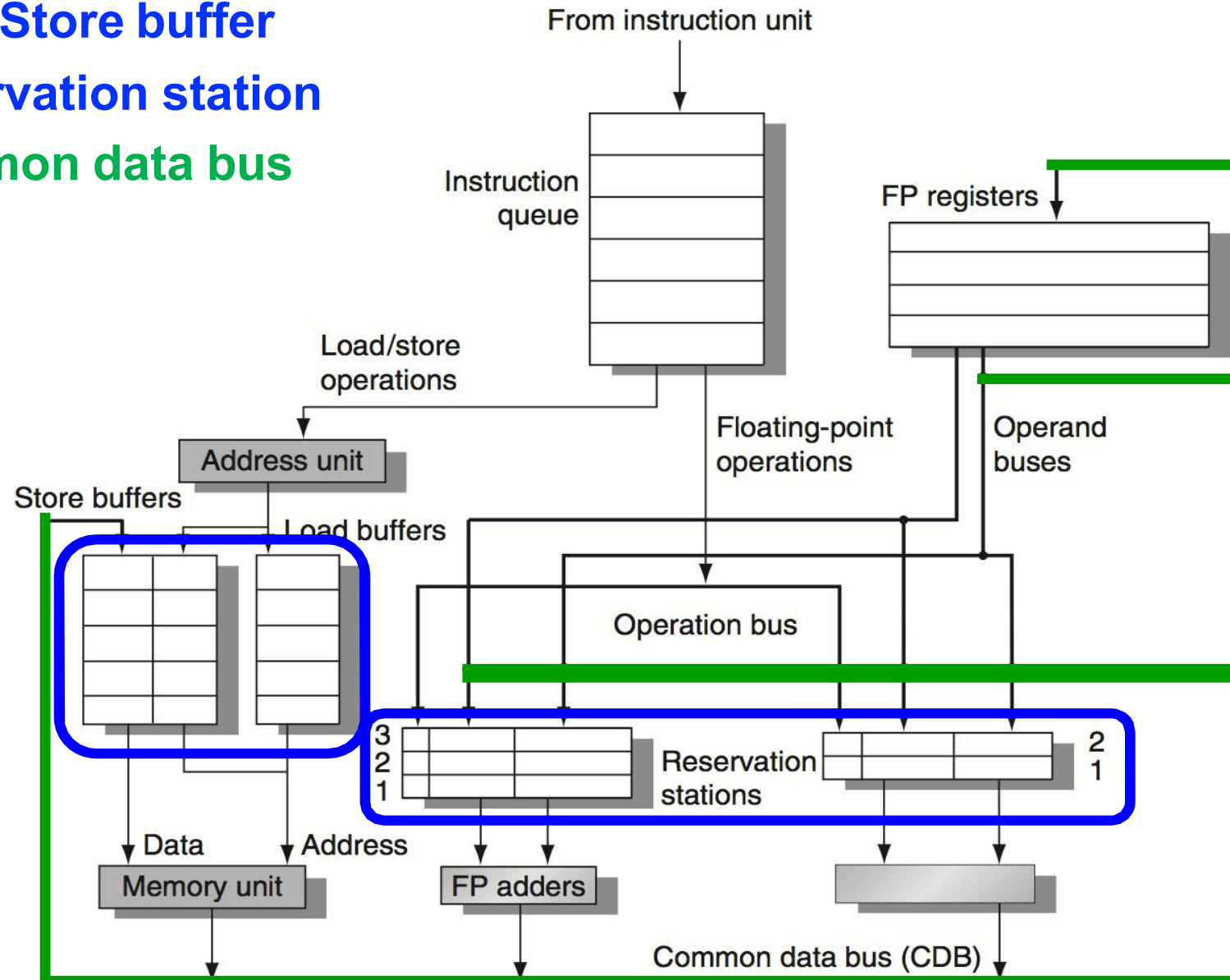
Clock	F0	F2	F4	F6	F8	F10	F12	...	F30
3									
	FU	Mult1	Load2		Load1				

- Note: registers names are removed ("renamed") in Reservation Stations

Organizations of Tomasulo's Algorithm in RISC-V



- Load/Store buffer
- Reservation station
- Common data bus



Details of Reservation Stations & Register Renaming



In Tomasulo's scheme, register renaming is provided by reservation stations, which buffer the operands of instructions waiting to issue and are associated with the functional units. The basic idea is that a reservation station fetches and buffers an operand as soon as it is available, eliminating the need to get the operand from a register. In addition, pending instructions designate the reservation station that will provide their input. Finally, when successive writes to a register overlap in execution, only the last one is actually used to update the register. As instructions are issued, the register specifiers for pending operands are renamed to the names of the reservation station, which provides register renaming.

Because there can be more reservation stations than real registers, the technique can even eliminate hazards arising from name dependences that could not be eliminated by a compiler. As we explore the components of Tomasulo's scheme, we will return to the topic of register renaming and see exactly how the renaming occurs and how it eliminates WAR and WAW hazards.

The use of reservation stations, rather than a centralized register file, leads to two other important properties. First, hazard detection and execution control are distributed: the information held in the reservation stations at each functional unit determines when an instruction can begin execution at that unit. Second, results are passed directly to functional units from the reservation stations where they are buffered, rather than going through the registers. This bypassing is done with a common result bus that allows all units waiting for an operand to be loaded simultaneously (on the 360/91, this is called the common data bus, or CDB). In pipelines that issue multiple instructions per clock and also have multiple execution units, more than one result bus will be needed.

Basic Architecture of Tomasulo

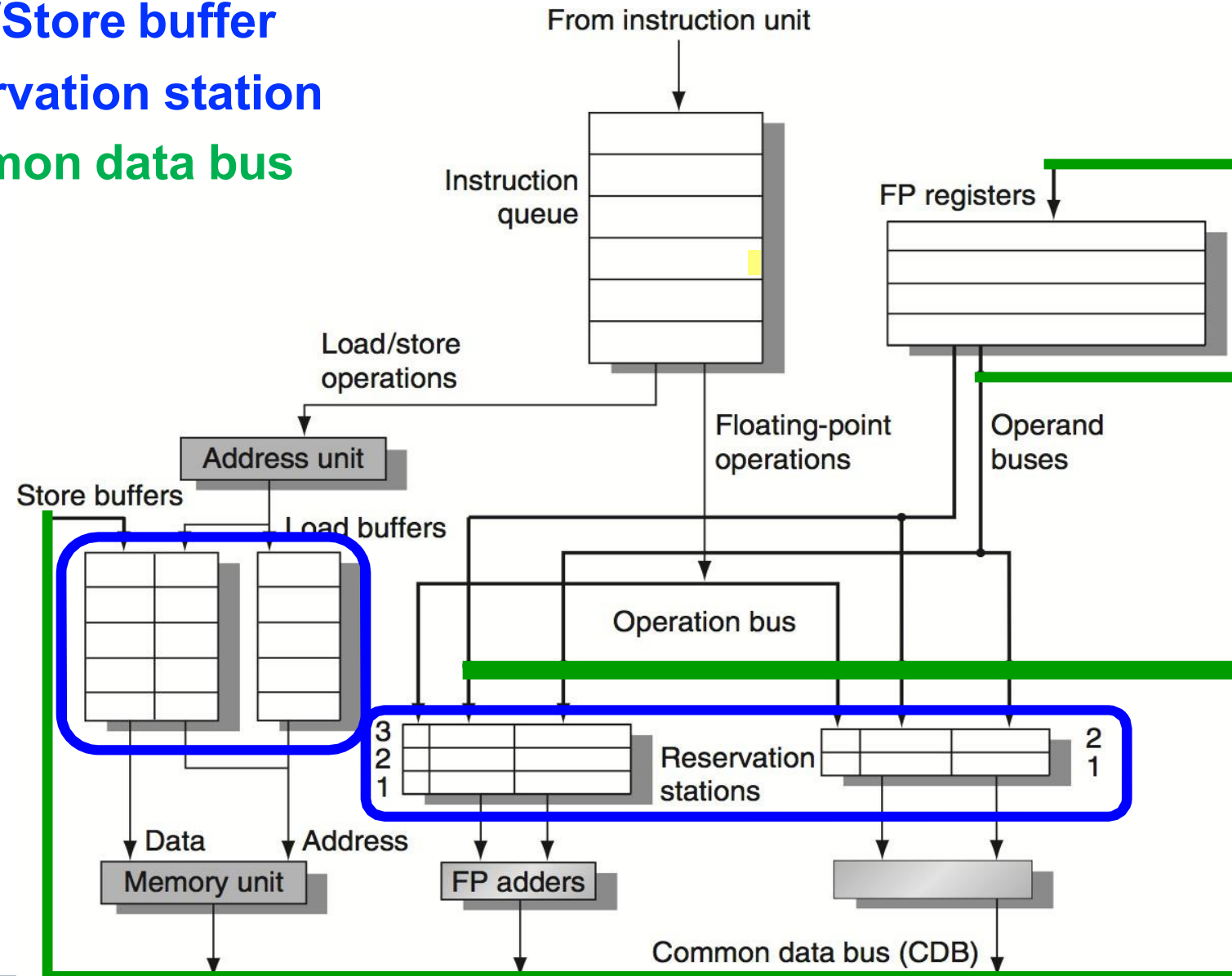
Figure 3.10 shows the basic structure of a Tomasulo-based processor, including both the floating-point unit and the load/store unit; none of the execution control tables is shown. Each reservation station holds an instruction that has been issued and is awaiting execution at a functional unit. If the operand values for that instruction have been computed, they are also stored in that entry; otherwise, the reservation station entry keeps the names of the reservation stations that will provide the operand values.

The load buffers and store buffers hold data or addresses coming from and going to memory and behave almost exactly like reservation stations, so we distinguish them only when necessary. The floating-point registers are connected by a pair of buses to the functional units and by a single bus to the store buffers. All results from the functional units and from memory are sent on the common data bus, which goes everywhere except to the load buffer. All reservation stations have tag fields, employed by the pipeline control.

CDB

Organizations of Tomasulo's Algorithm in RISC-V

- Load/Store buffer
- Reservation station
- Common data bus



Structure of Tomasulo

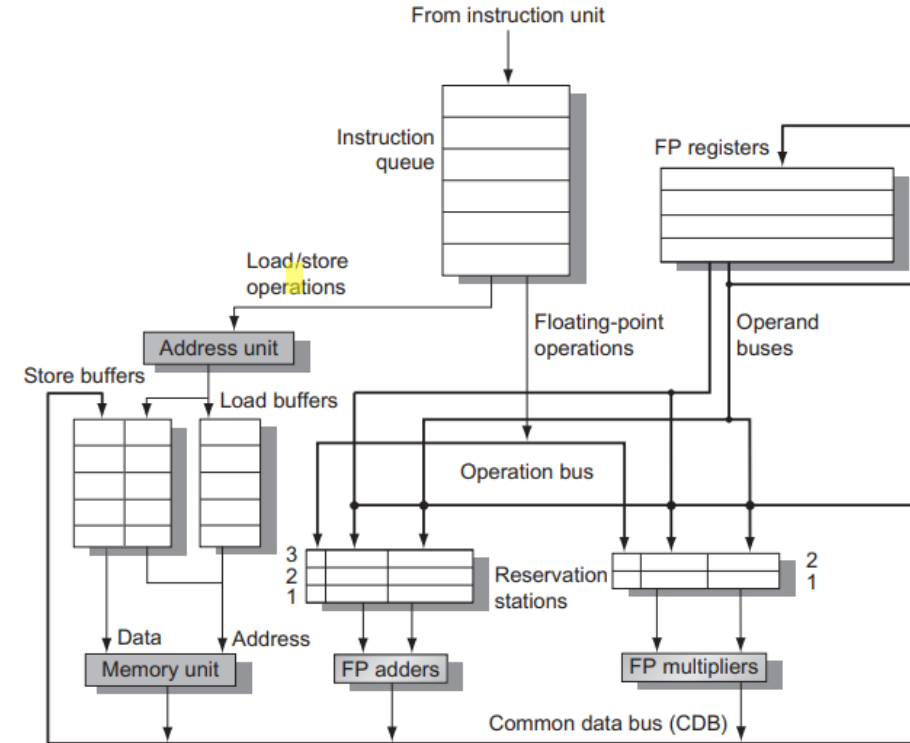


Figure 3.10 The basic structure of a RISC-V floating-point unit using Tomasulo's algorithm. Instructions are sent from the instruction unit into the instruction queue from which they are issued in first-in, first-out (FIFO) order. The reservation stations include the operation and the actual operands, as well as information used for detecting and resolving hazards. Load buffers have three functions: (1) hold the components of the effective address until it is computed, (2) track outstanding loads that are waiting on the memory, and (3) hold the results of completed loads that are waiting for the CDB. Similarly, store buffers have three functions: (1) hold the components of the effective address until it is computed, (2) hold the destination memory addresses of outstanding stores that are waiting for the data value to store, and (3) hold the address and value to store until the memory unit is available. All results from either the FP units or the load unit are put on the CDB, which goes to the FP register file as well as to the reservation stations and store buffers. The FP adders implement addition and subtraction, and the FP multipliers do multiplication and division.

Three Stages of Tomasulo Algorithm



1. **Issue**—get instruction from FP Op Queue

If reservation station free (no structural hazard),
control issues instr & sends operands (renames registers).

2. **Execution**—operate on operands (EX)

When both operands ready then execute;
if not ready, watch Common Data Bus for result

3. **Write result**—finish execution (WB)

Write on Common Data Bus to all awaiting units;
mark reservation station available

- Normal data bus: data + destination (“go to” bus)
- Common data bus: data + source (“come from” bus)
 - 64 bits of data + 4 bits of Functional Unit source address
 - Write if matches expected Functional Unit (produces result)
 - Does the broadcast

CDB Bus + Reservation Station + Tags

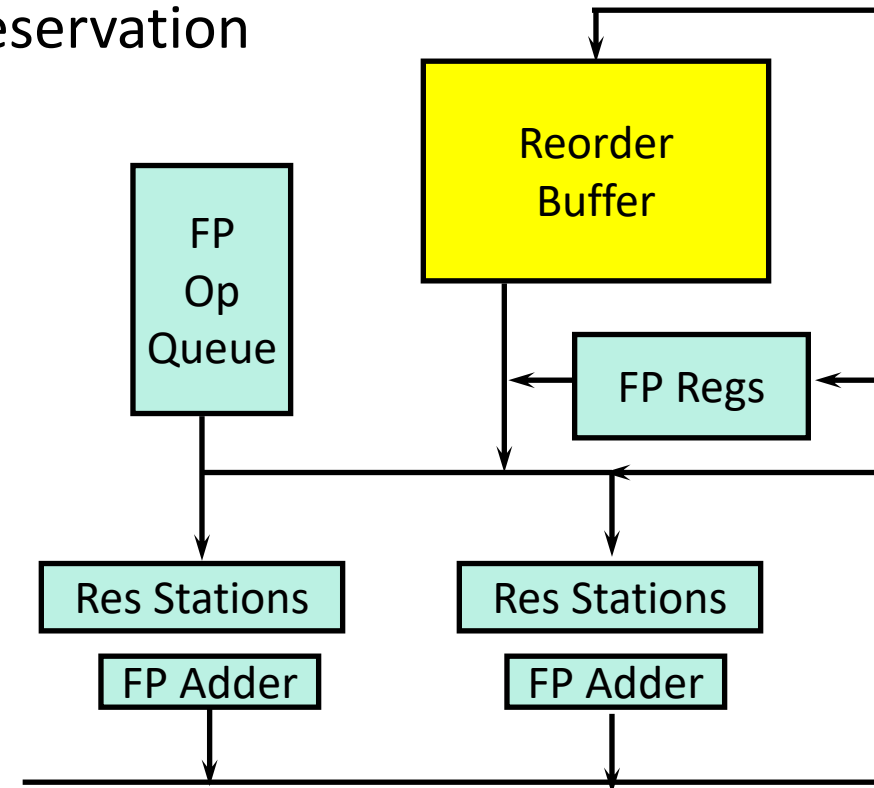


In Tomasulo's scheme, as well as the subsequent methods we look at for supporting speculation, results are broadcast on a bus (the CDB), which is monitored by the reservation stations. The combination of the common result bus and the retrieval of results from the bus by the reservation stations implements the forwarding and bypassing mechanisms used in a statically scheduled pipeline. In doing so, however, a dynamically scheduled scheme, such as Tomasulo's algorithm, introduces one cycle of latency between source and result because the matching of a result and its use cannot be done until the end of the Write Result stage, as opposed to the end of the Execute stage for a simpler pipeline. Thus, in a dynamically scheduled pipeline, the effective latency between a producing instruction and a consuming instruction is at least one cycle longer than the latency of the functional unit producing the result.

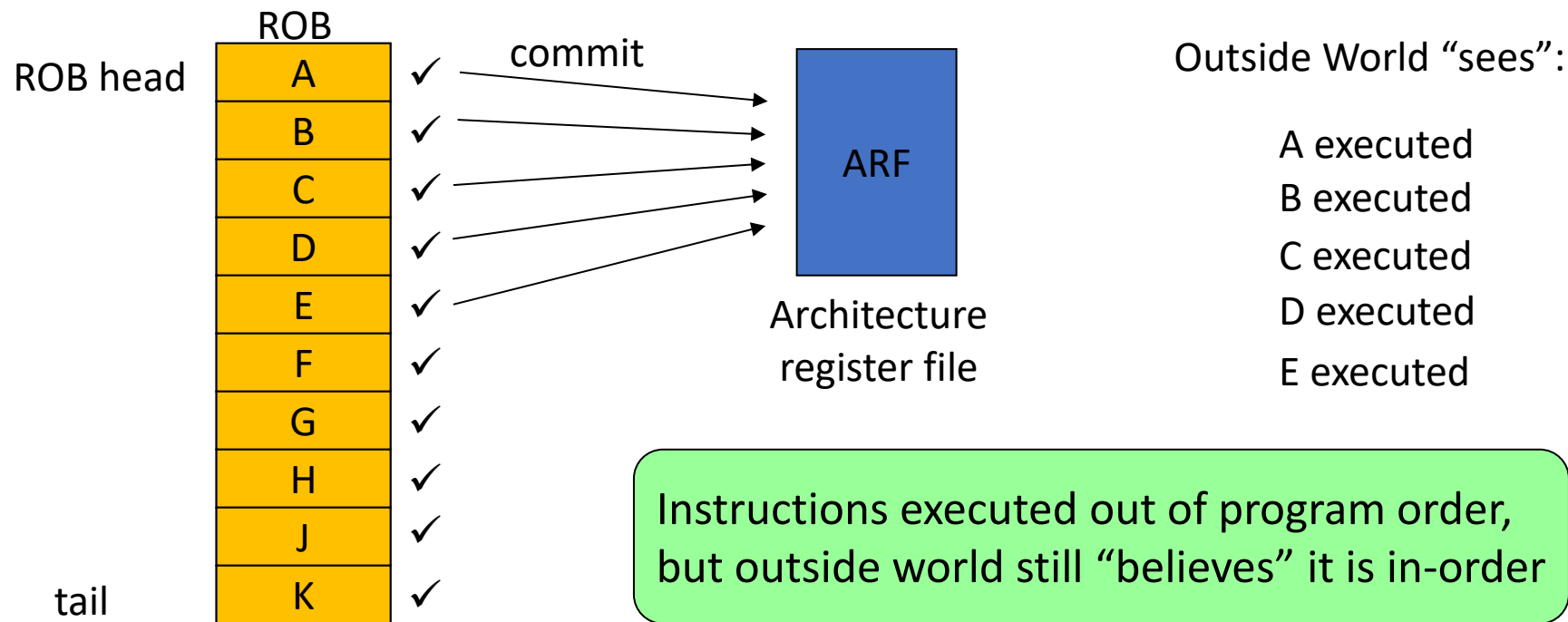
It is important to remember that the tags in the Tomasulo scheme refer to the buffer or unit that will produce a result; the register names are discarded when an instruction issues to a reservation station (This is a key difference between Tomasulo's scheme and scoreboarding: in scoreboarding, operands stay in the registers and are read only after the producing instruction completes and the consuming instruction is ready to execute.)

ReOrder Buffer (ROB)

- HW buffer for results of uncommitted instructions
 - At least 3 fields: instruction, destination, value
 - Can be operand source → virtual registers
 - Use reorder buffer # instead of reservation station # as tag ID on CDB
 - Supplies operands between execution complete and commit
 - After instruction commits, result is put into register
 - Easy to undo speculated instr. on mispredicted branches or exceptions

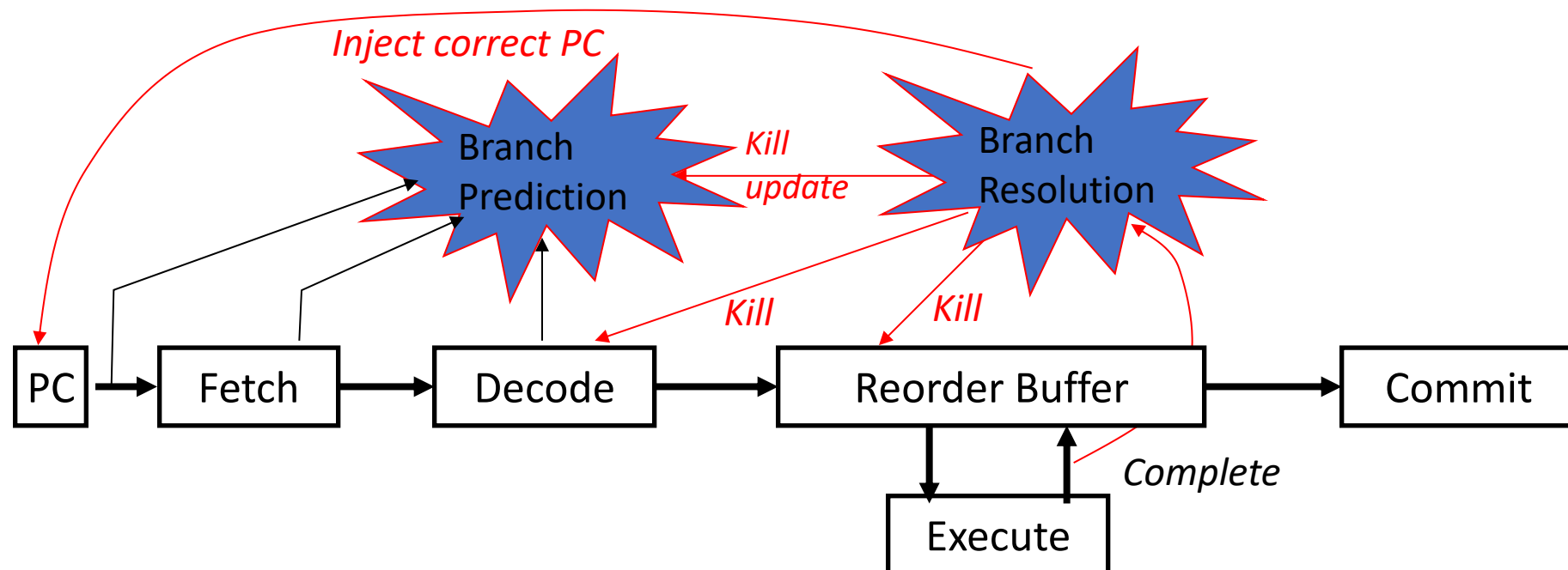


- New step for making instruction execution “visible” to the outside world
 - It “commits” the changes to the architectural state

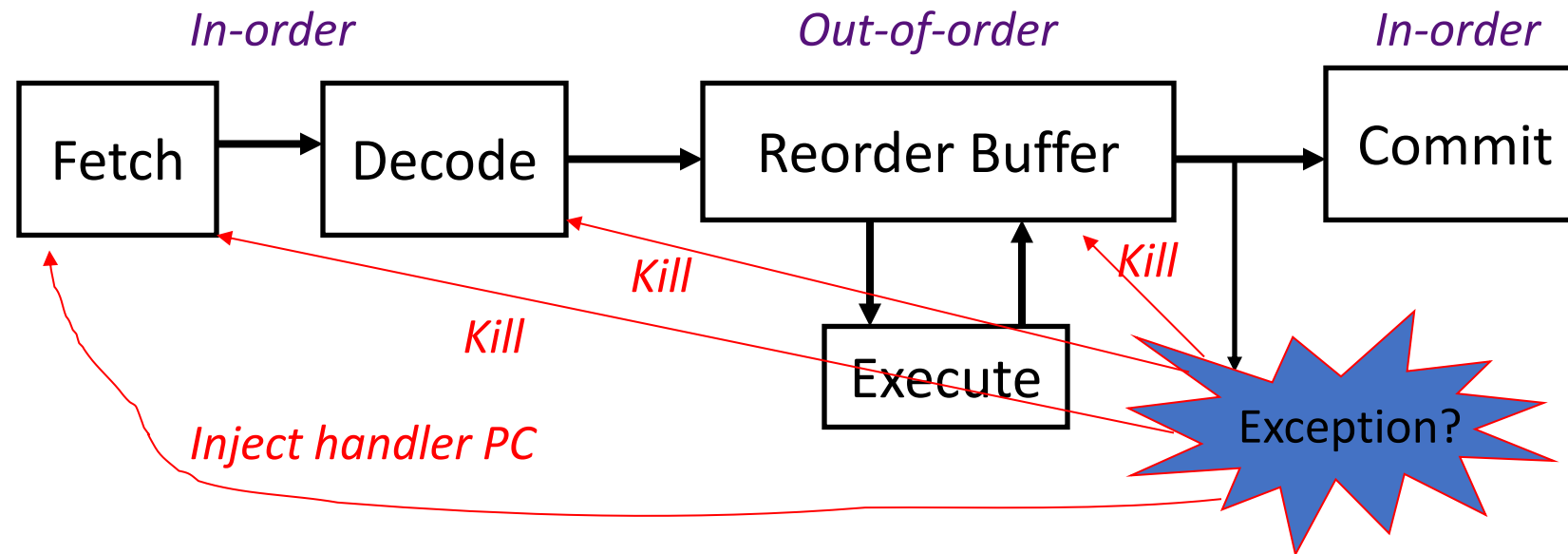


Handling Incorrect Speculation

- Instructions following mispredicted branch, i.e. those in decode/issue buffers & RSs, are invalidated
- ROB entries of these instructions are deallocated
- Restart fetch at correct branch successor



Handling Precise Exception / Interrupt



- Instructions fetched and decoded into ROB in-order
- Execution is out-of-order → OOO completion
- Commit (write-back to architectural state, i.e., register file and memory) is in-order

Example to introduce Tomasulo Control Fields

Example to demonstrate Tomasulo control fields

Floating pt. 0-0-0

LD	F6	34+	R2
LD	F2	45+	R3
MULTD	F0	F2	F4
SUBD	F8	F6	F2
DIVD	F10	F0	F6
ADDD	F6	F8	F2

IBM 360/91 used Issue, Execute, WB

⇒ With a different reservation station field structure

Indicating instruction and register status

Assume

→ 3 FP Adder R.S.

→ 2 FP Mult R.S.

- Load – 2 cycles
- Add 2 cycles
- MUL 10 cycles
- Div 40 cycles to execute

Seven Fields in Reservation Station

Each reservation station has seven fields:

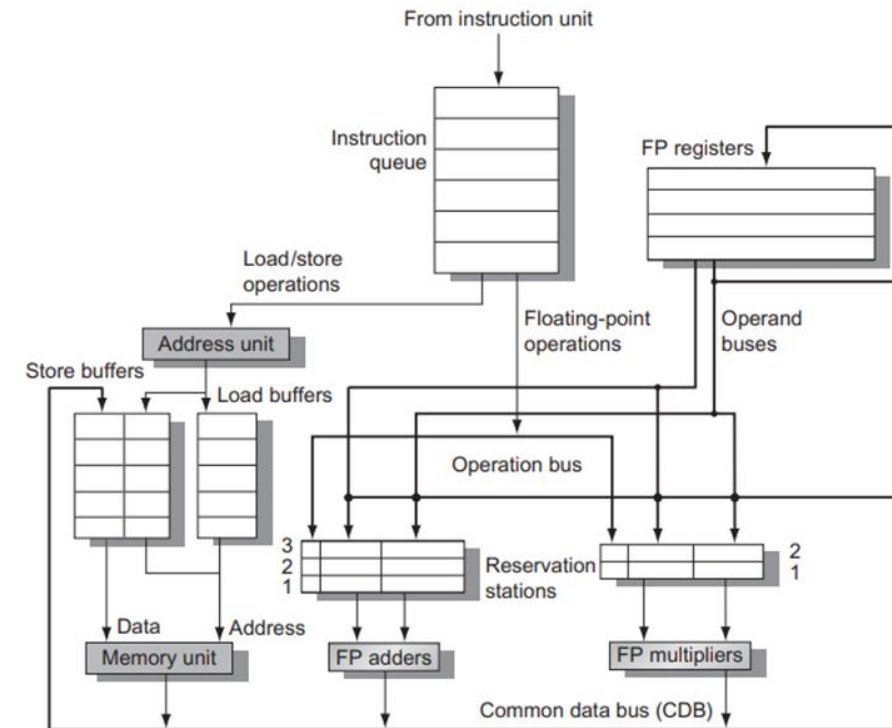
- **Op**—The operation to perform on source operands S1 and S2.
- **Qj, Qk**—The reservation stations that will produce the corresponding source operand; a value of zero indicates that the source operand is already available in Vj or Vk, or is unnecessary.
- **Vj, Vk**—The value of the source operands. Note that only one of the V fields or the Q field is valid for each operand. For loads, the Vk field is used to hold the offset field.
- **A**—Used to hold information for the memory address calculation for a load or store. Initially, the immediate field of the instruction is stored here; after the address calculation, the effective address is stored here.
- **Busy**—Indicates that this reservation station and its accompanying functional unit are occupied.

The register file has a field, Qi:

- **Qi**—The number of the reservation station that contains the operation whose result should be stored into this register. If the value of Qi is blank (or 0), no currently active instruction is computing a result destined for this register, meaning that the value is simply the register contents.

The load and store buffers each have a field, A, which holds the result of the effective address once the first step of execution has been completed.

In the next section, we will first consider some examples that show how these mechanisms work and then examine the detailed algorithm.



Info in Reservation Station

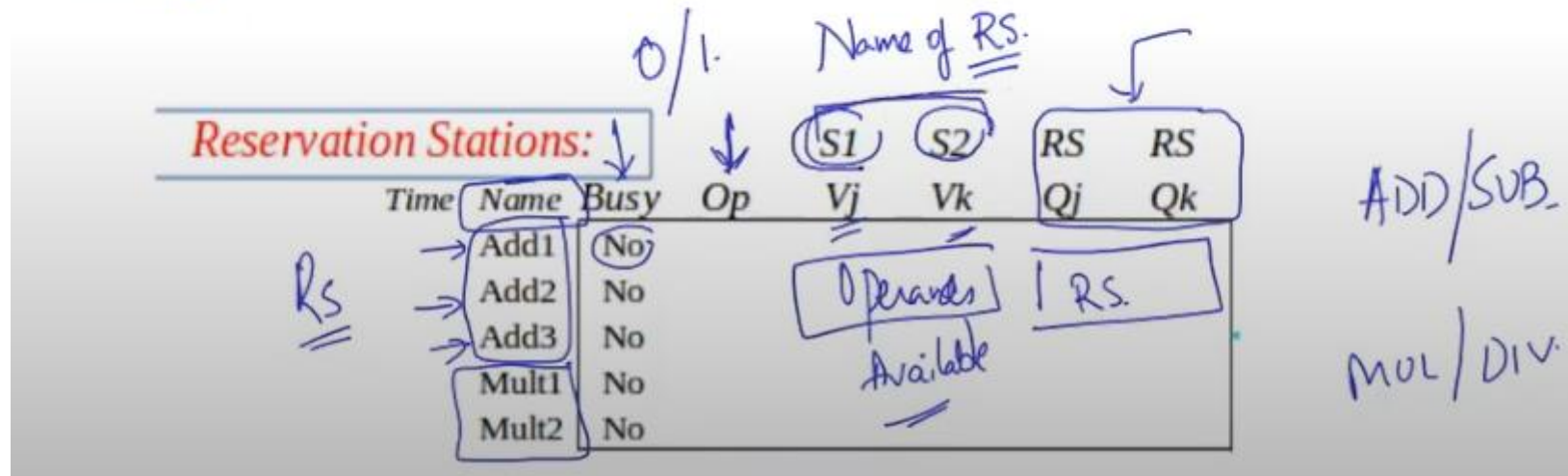
Reservation Station Components

Op: Operation to perform in the unit (e.g., + or −)

V_j, V_k : Value of Source operands

Q_j, Q_k : Reservation stations producing source registers (value to be written)

Busy: Indicates reservation station or FU is busy



Tomasulo Starting Status

Tomasulo Example

Instruction status:

Instruction		<i>j</i>	<i>k</i>	Issue	Exec Comp	Write Result	Load1	Load2	Load3	Busy	Address
LD	F6	34+	R2							No	
LD	F2	45+	R3							No	
MULTD	F0	F2	F4							No	
SUBD	F8	F6	F2							No	
DIVD	F10	F0	F6							No	
ADDD	F6	F8	F2							No	

3 Load/Buffers

Reservation Stations:

Time	Name	Busy	Op	<i>S1</i> <i>Vj</i>	<i>S2</i> <i>Vk</i>	<i>RS</i> <i>Qj</i>	<i>RS</i> <i>Qk</i>
	Add1	No					
	Add2	No					
	Add3	No					
	Mult1	No					
	Mult2	No					

3 FP Adder R.S.
2 FP Mult R.S.

Register result status:

Clock		<i>F0</i>	<i>F2</i>	<i>F4</i>	<i>F6</i>	<i>F8</i>	<i>F10</i>	<i>F12</i>
0								

Clock cycle counter

Vj, *Vk*: Value of Source operands

Qj, *Qk*: Reservation stations producing source registers values

7

Tomasulo Example Cycle 1

{ In-order / SNE-
 { 0-0-0 WB → No ROB-

Tomasulo Example Cycle 1

Instruction status:

Instruction	<i>j</i>	<i>k</i>	Issue	Exec	Write
LD	F6	34+	R2	1	
LD	F2	45+	R3		
MULTD	F0	F2	F4		
SUBD	F8	F6	F2		
DIVD	F10	F0	F6		
ADDD	F6	F8	F2		

	Busy	Address
Load1	Yes	34+R2
Load2	No	
Load3	No	

Mem-

→ H&P show Load also as part of Rs, with an additional 'A' field

Reservation Stations:

Time	Name	Busy	Op	<i>S1</i> <i>Vj</i>	<i>S2</i> <i>Vk</i>	<i>RS</i> <i>Qj</i>	<i>RS</i> <i>Qk</i>
	Add1	No					
	Add2	No					
	Add3	No					
	Mult1	No					
	Mult2	No					

Issue Load 1 --> load buffer

Register result status:

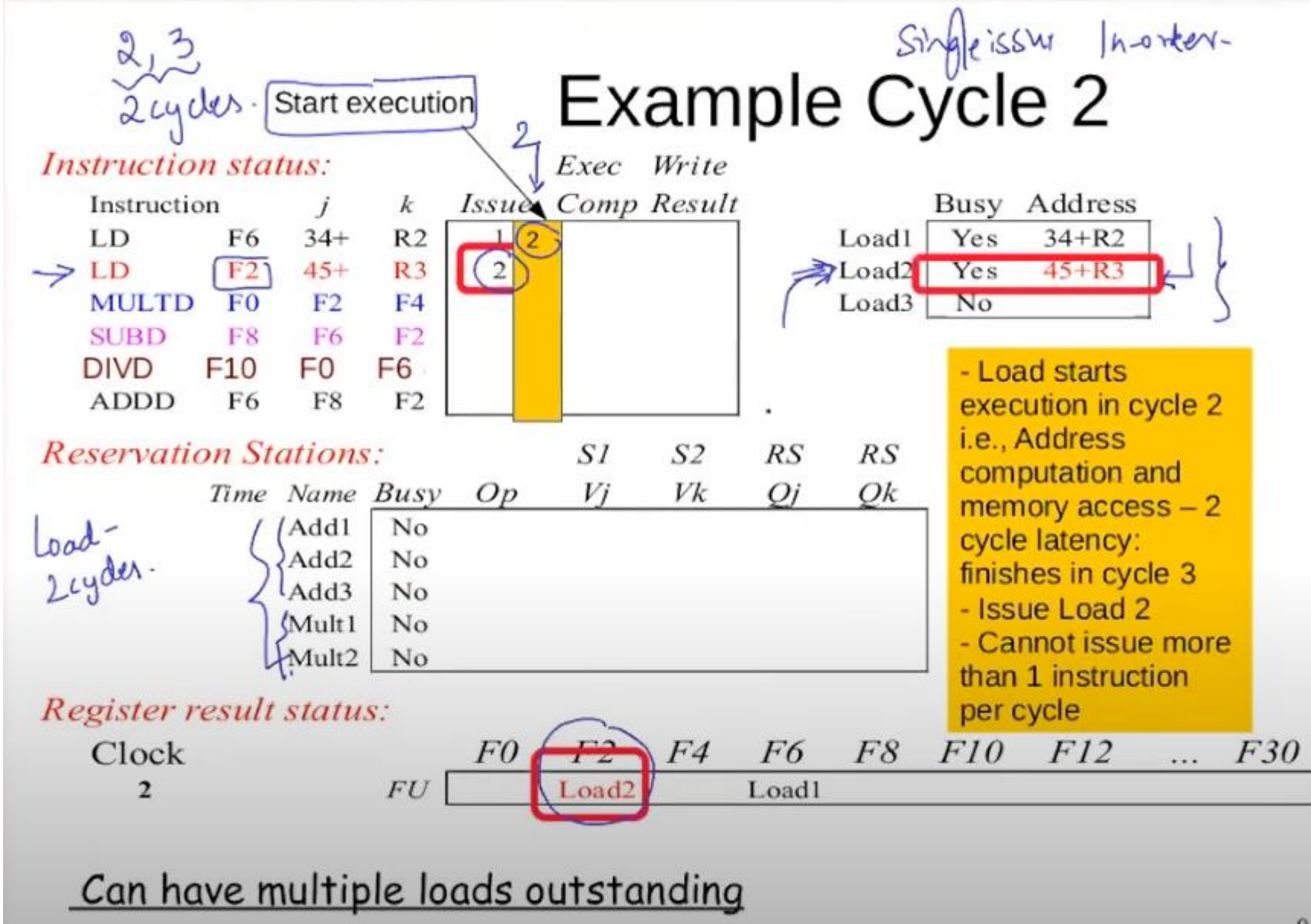
Clock	F0	F2	F4	F6	F8	F10	F12	...	F30
1				Load1					

F6 - Rs. Load!

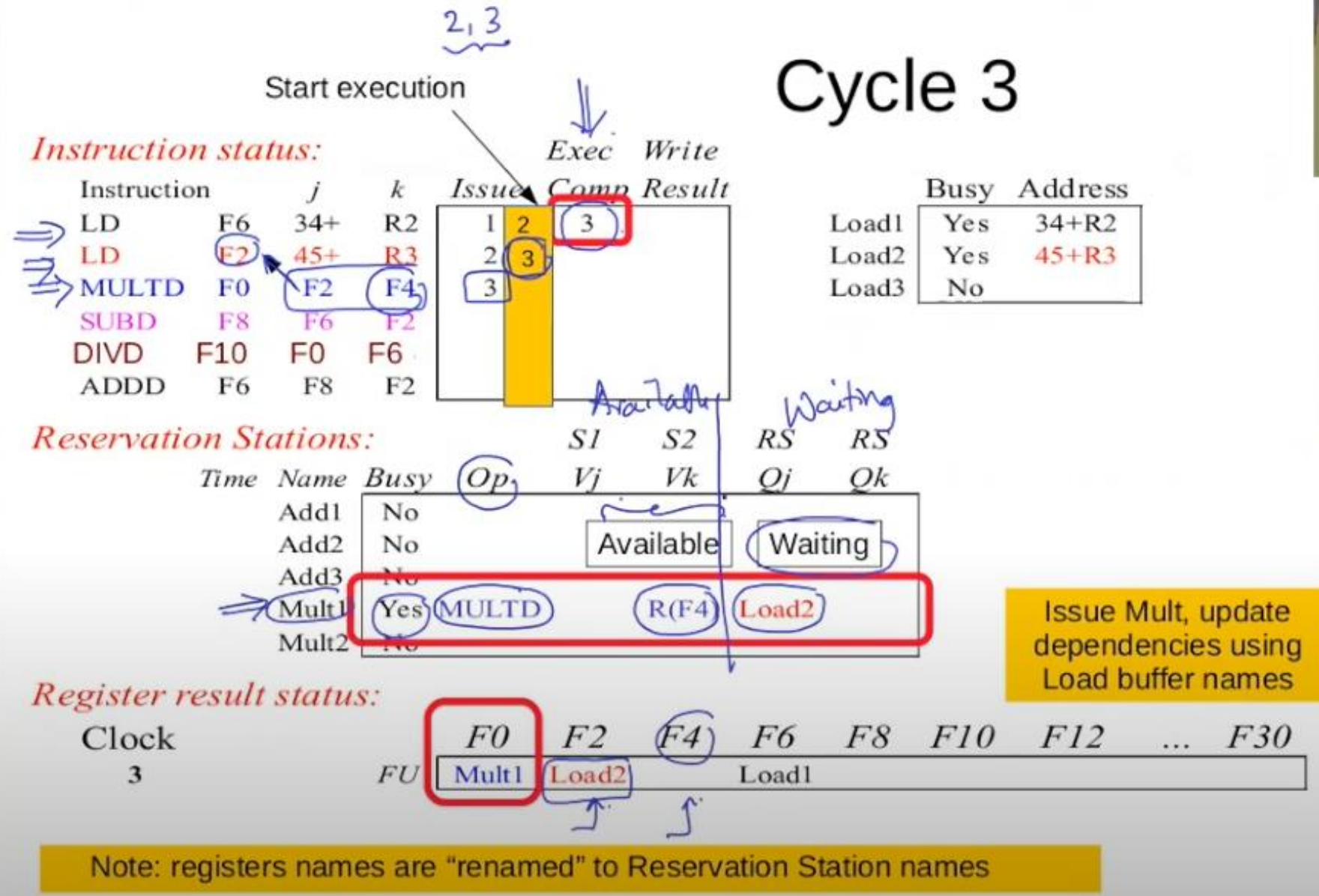
⇒ Note: registers names are "renamed" to Reservation Station names

Note that we are not showing the Fetch and Decode cycles here. We start directly from Issuing the instruction

Tomasulo Example Cycle 2



Tomasulo Example Cycle 3



Tomasulo Example Cycle 3

Instruction status:

Instruction	j	k	Exec Write		Issue	Comp	Result		Busy	Address
LD	F6	34+	R2		1	3		Load1	Yes	34+R2
LD	F2	45+	R3		2			Load2	Yes	45+R3
MULTD	F0	F2	F4		3			Load3	No	
SUBD	F8	F6	F2							
DIVD	F10	F0	F6							
ADDD	F6	F8	F2							

Reservation Stations:

Time	Name	Busy	Op	$S1$		$S2$		RS	
				V_j	V_k	Q_j	Q_k		
	Add1	No							
	Add2	No							
	Add3	No							
	Mult1	Yes	MULTD		R(F4)		Load2		
	Mult2	No							

Register result status:

Clock	F0	F2	F4	F6	F8	F10	F12	...	F30
3	FU	Mult1	Load2		Load1				

- Note: registers names are removed ("renamed") in Reservation Stations

Example Show what the information tables look like for the following code sequence when only the first load has completed and written its result:

```
1. fld      f6,32(x2)
2. fld      f2,44(x3)
3. fmul.d   f0,f2,f4
4. fsub.d   f8,f2,f6
5. fdiv.d   f0,f0,f6
6. fadd.d   f6,f8,f2
```

← completed

Answer Figure 3.11 shows the result in three tables. The numbers appended to the names Add, Mult, and Load stand for the tag for that reservation station—Add1 is the tag for the result from the first add unit. In addition, we have included an instruction status table. This table is included only to help you understand the algorithm; it is *not* actually a part of the hardware. Instead, the reservation station keeps the state of each operation that has issued.

For solving - Seven Fields in Reservation Station

Each reservation station has seven fields:

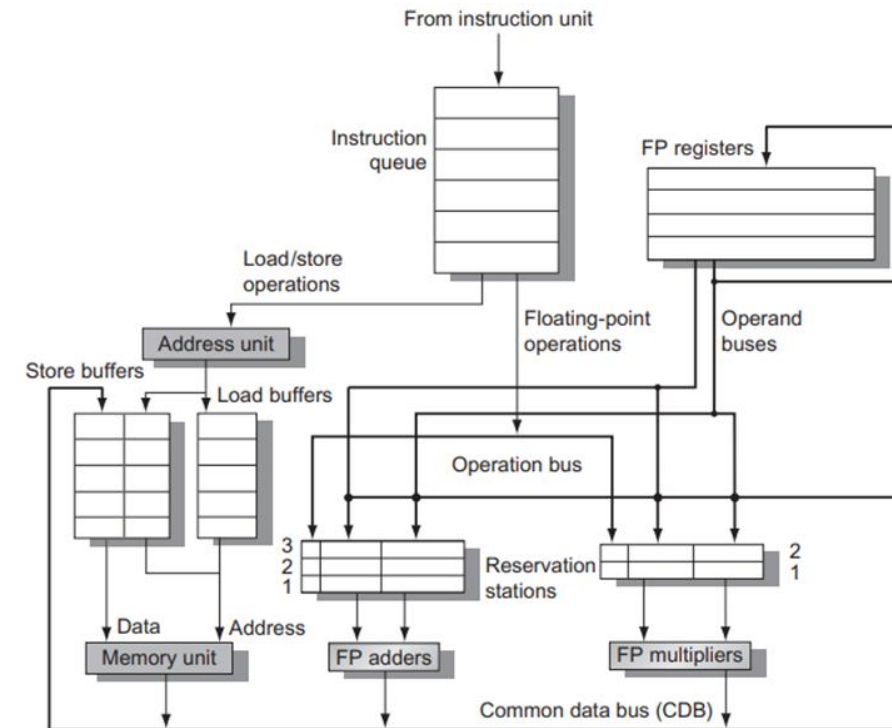
- **Op**—The operation to perform on source operands S1 and S2.
- **Qj, Qk**—The reservation stations that will produce the corresponding source operand; a value of zero indicates that the source operand is already available in Vj or Vk, or is unnecessary.
- **Vj, Vk**—The value of the source operands. Note that only one of the V fields or the Q field is valid for each operand. For loads, the Vk field is used to hold the offset field.
- **A**—Used to hold information for the memory address calculation for a load or store. Initially, the immediate field of the instruction is stored here; after the address calculation, the effective address is stored here.
- **Busy**—Indicates that this reservation station and its accompanying functional unit are occupied.

The register file has a field, Qi:

- **Qi**—The number of the reservation station that contains the operation whose result should be stored into this register. If the value of Qi is blank (or 0), no currently active instruction is computing a result destined for this register, meaning that the value is simply the register contents.

The load and store buffers each have a field, A, which holds the result of the effective address once the first step of execution has been completed.

In the next section, we will first consider some examples that show how these mechanisms work and then examine the detailed algorithm.



		Instruction status		
Instruction		Issue	Execute	Write result
f1d	f6,32(x2)	✓	✓	✓
f1d	f2,44(x3)	✓	✓	
fmul.d	f0,f2,f4	✓		
fsub.d	f8,f2,f6	✓		
fdiv.d	f0,f0,f6	✓		
fadd.d	f6,f8,f2	✓		

Reservation stations							
Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load1	No						
Load2	Yes	Load					44 + Regs[x3]
Add1	Yes	SUB		Mem[32 + Regs[x2]]	Load2		
Add2	Yes	ADD			Add1	Load2	
Add3	No						
Mult1	Yes	MUL		Regs[f4]	Load2		
Mult2	Yes	DIV		Mem[32 + Regs[x2]]	Mult1		

Register status									
Field	f0	f2	f4	f6	f8	f10	f12	...	f30
Qi	Mult1	Load2		Add2	Add1	Mult2			

Figure 3.11 Reservation stations and register tags shown when all of the instructions have issued but only the first load instruction has completed and written its result to the CDB. The second load has completed effective address calculation but is waiting on the memory unit. We use the array Regs[] to refer to the register file and the array Mem[] to refer to the memory. Remember that an operand is specified by either a Q field or a V field at any time. Notice that the fadd.d instruction, which has a WAR hazard at the WB stage, has issued and could complete before the fdiv.d initiates.

Example Using the same code segment as in the previous example (page 201), show what the status tables look like when the `fmul.d` is ready to write its result.

Answer The result is shown in the three tables in Figure 3.12. Notice that `fadd.d` has completed because the operands of `fdiv.d` were copied, thereby overcoming the WAR hazard. Notice that even if the load of `f6` was `fdiv.d`, the add into `f6` could be executed without triggering a WAW hazard.

1.	<code>fld</code>	<code>f6, 32(x2)</code>
2.	<code>fld</code>	<code>f2, 44(x3)</code>
3.	<code>fmul.d</code>	<code>f0, f2, f4</code>
4.	<code>fsub.d</code>	<code>f8, f2, f6</code>
5.	<code>fdiv.d</code>	<code>f0, f0, f6</code>
6.	<code>fadd.d</code>	<code>f6, f8, f2</code>

Solution



Instruction	Instruction status		
	Issue	Execute	Write result
f1d f6,32(x2)	✓	✓	✓
f1d f2,44(x3)	✓	✓	✓
fmul.d f0,f2,f4	✓	✓	
fsub.d f8,f2,f6	✓	✓	✓
fdiv.d f0,f0,f6	✓		
fadd.d f6,f8,f2	✓	✓	✓

Reservation stations						
Name	Busy	Op	Vj	Vk	Qj	Qk
Load1	No					
Load2	No					
Add1	No					
Add2	No					
Add3	No					
Mult1	Yes	MUL	Mem[44 + Regs[x3]]	Regs[f4]		
Mult2	Yes	DIV		Mem[32 + Regs[x2]]	Mult1	

Register status									
Field	f0	f2	f4	f6	f8	f10	f12	...	f30
Qi	Mult1					Mult2			

Figure 3.12 Multiply and divide are the only instructions not finished.

Differences between Scoreboard and Tomasulo

Tomasulo vs Scoreboard – Key Differences

→ CDB broadcasts results to FU

→ load & stores treated as FU as well

Stages: Issue, Execution & write result.

→ window size: ≤ 14 inst

→ WAR dependency: ~~renaming~~ renaming avoids
WAW " →

→ ~~Results forwarding~~ → Brc

→ CS → distributed R.S

forwarding → write/read reg.

→ stages:- Issue
Read operands
Execution
Write results.

→ window size: ≤ 5 inst

→ WAR dependency: stall completion.
WAW " →

→ control structure: Central scoreboard.

Scoreboard Example

Reference:

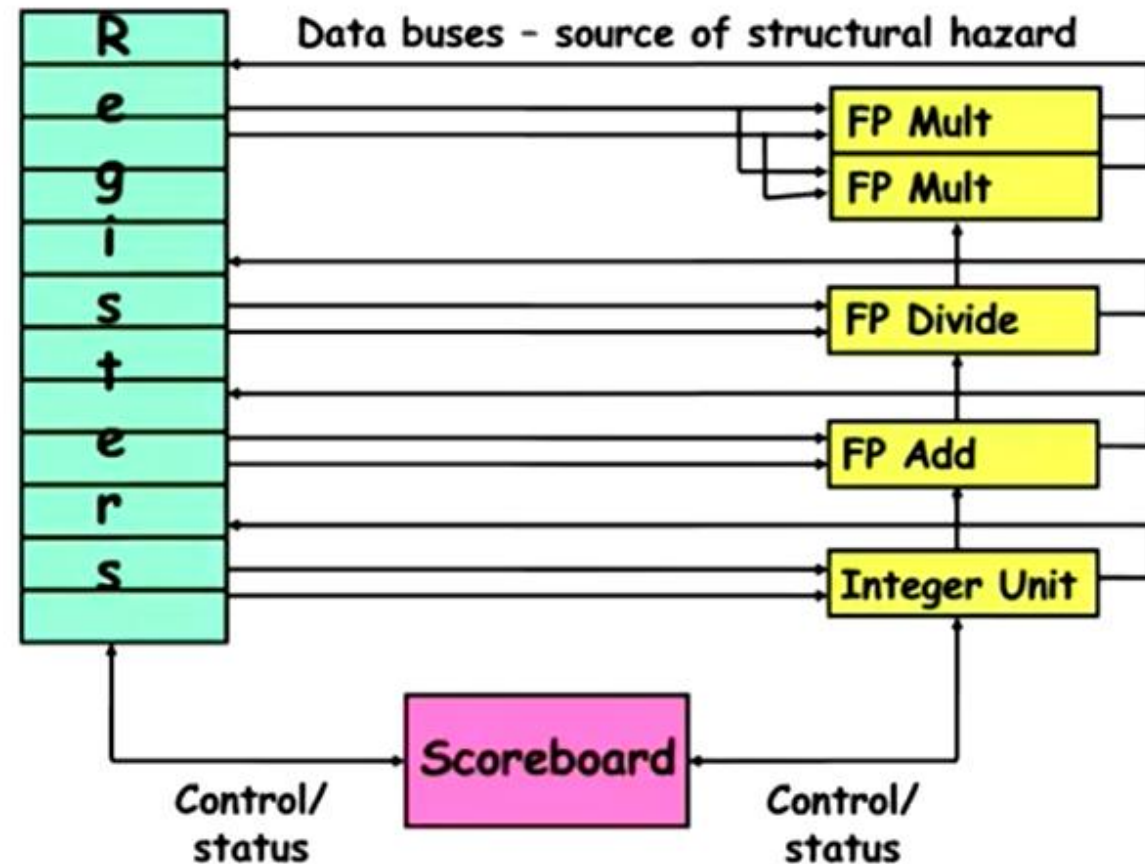
<https://www.youtube.com/watch?v=TJcNJh8tS6w>

A Scoreboard For MIPS :

Every instruction goes through a centralised hardware scoreboard. The scoreboard sends control signal to tell when an instruction can execute and when it can write results into destination register.

Out of order execution requires multiple instruction in the EX stage simultaneously which can be achieved with multiple functional units.

The instructions having a dependence are stalled but those having no dependence are allowed to continue.



3 Parts Of Scoreboard :

1. Instruction Status : In which of the 4 stages(Issue,RO,EX,WB) the instruction is.

2.Functional Unit Status :

- i. Busy :Whether the functional unit is busy or not.
- ii. Op : Operation to be performed.
- iii. F_i : Destination Register.
- iv. F_j, F_k : Source registers.
- v. Q_j, Q_k : Functional unit going to produce source registers F_j and F_k
- vi. R_j, R_k : Flags indicating whether F_j and F_k are ready or not.

3. Register Result Status : Indicates which functional unit will write the registers.

4 Stages of Scoreboard Control

4 Stages of Scoreboard Control :

1. (ID Stage 1) Issue : Here instruction is decoded. If the functional unit is free (ie. no structural hazard), and no previously stalled instruction has the same destination register (ie. no WAW dependency), the scoreboard issues the instruction.

Issue of instruction always takes place in order . If there is structural or WAW hazard then the instruction issue stalls and no further instruction will be issued until these hazards are cleared.

2. (ID Stage 2) Read Operand : A source operand is available or we can read a source operand if no earlier issued stalled instructions are going to write it (ie. the scoreboard resolves RAW hazard dynamically in this stage). When the source operand is available, the scoreboard tells the functional units to proceed to read operands from the registers and begin execution. The instructions may be sent to execution stage out of order.

3. Execution (EX) : Operate on operands. When result is ready control signal is sent to the scoreboard.

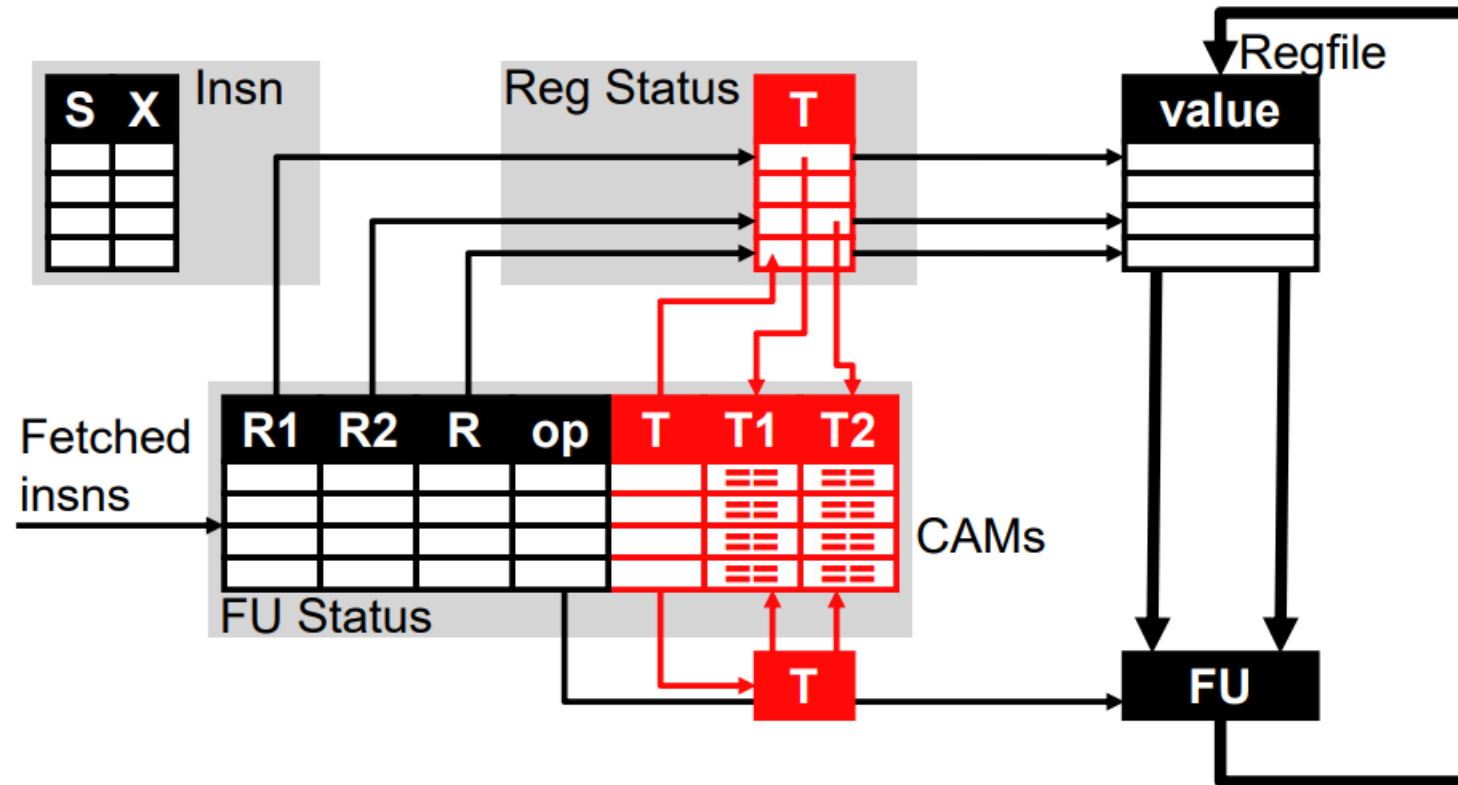
4. Write Back (WB) : Once the scoreboard is aware that the functional unit has completed its execution, it checks for WAR hazard. If none, it writes the result. If WAR hazard exists, then it stalls the instruction.

Eg : Div F0 F2 F4
Add F6 F2 F8
Sub F8 F10 F12

Let EX stage of Sub instruction is completed before Add instruction, but to resolve WAR hazard for F8, the sub instruction will be stalled until Add instruction does not read the data from F8.

Scoreboard Structure

Simple Scoreboard Data Structures



- Insn fields and status bits
- **Tags**
- Values

Question on Scoreboard



Q. Write the steps of execution with scoreboard approach. The following set of MIPS instruction is going to be executed in a pipelined system.

LD F6, 34(R2)
LD F2, 45(R3)
MUL F0, F2, F4
SUB F8, F6, F2
DIV F10, F0, F6
ADD F6, F8, F2

- Reference:
- <https://www.youtube.com/watch?v=TJcNJh8tS6w>

The latencies of - Integer unit is 1 cycle
 FP adder unit is 2 cycle
 FP multiplier is 10 cycle
 FP divider unit is 40 cycles

Starting with 1st Instruction

1. Instruction Status :

Instr ⁿ .	i	j	k	Issue	RO	EX	WB
LD F6, 34(R2)							
LD F2, 45(R3)							
MUL F0, F2, F4							
SUB F8, F6, F2							
DIV F10, F0, F6							
ADD F6, F8, F2							

2. Functional Unit Status :

FU	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer									
Mult1									
Mult2									
Add									
Div									

1. Instruction Status : In which of the 4 stages(Issue,RO,EX,WB) the instruction is.

2. Functional Unit Status :

- i. Busy :Whether the functional unit is busy or not.
- ii. Op : Operation to be performed.
- iii. F_i : Destination Register.
- iv. F_j, F_k : Source registers.
- v. Q_j, Q_k : Functional unit going to produce source registers F_j and F_k
- vi. R_j, R_k : Flags indicating whether F_j and F_k are ready or not.

3. Register Result Status : Indicates which functional unit will write the registers.

3. Register Result Status :

	F0	F2	F4	F6	F8	F10	F12	F14
FU								

Cycle 1

1. Instruction Status :

Instr ⁿ . i j k	Issue	RO	EX	WB
LD F6, 34(R2)	1			
LD F2, 45(R3)				
MUL F0, F2, F4				
SUB F8, F6, F2				
DIV F10, F0, F6				
ADD F6, F8, F2				

<http://www.flashbackrecorder.com/>
Watermark removed when fully licensed

3. Register Result Status :

	F0	F2	F4	F6	F8	F10	F12	F14
FU				I				

2. Functional Unit Status :

FU	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer	Yes	Load	F6		R2				Yes
Mult1									
Mult2									
Add									
Div									

1. Instruction Status :

Instr ⁿ . i j k	Issue	RO	EX	WB
LD F6, 34(R2)	1	2	3	4
LD F2, 45(R3)	I			
MUL F0, F2, F4				
SUB F8, F6, F2				
DIV F10, F0, F6				
ADD F6, F8, F2				

<http://www.flashbackrecorder.com/>
Watermark removed when fully licensed

3. Register Result Status :

	F0	F2	F4	F6	F8	F10	F12	F14
FU								

2. Functional Unit Status :

FU	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer									
Mult1									
Mult2									
Add									
Div									

1. Instruction Status :

Instr ⁿ . i j k	Issue	RO	EX	WB
LD F6, 34(R2)	1	2	3	4
LD F2, 45(R3)	5	I		
MUL F0, F2, F4				
SUB F8, F6, F2				
DIV F10, F0, F6				
ADD F6, F8, F2				

The latencies of - Integer unit is 1 cycle
 FP adder unit is 2 cycle
 FP multiplier is 10 cycle
 FP divider unit is 40 cycles

3. Register Result Status :

	F0	F2	F4	F6	F8	F10	F12	F14
FU		Int						

2. Functional Unit Status :

FU	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer	Yes	Load	F2		R3				Yes
Mult1									
Mult2									
Add									
Div									

1. Instruction Status :

Instr ⁿ . i j k	Issue	RO	EX	WB
LD F6, 34(R2)	1	2	3	4
LD F2, 45(R3)	5	6	I	
MUL F0, F2, F4	6			
SUB F8, F6, F2				
DIV F10, F0, F6				
ADD F6, F8, F2				

3. Register Result Status :

	F0	F2	F4	F6	F8	F10	F12	F14
FU	Mult1	Int						

2. Functional Unit Status :

FU	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer	Yes	Load	F2		R3				Yes
Mult1	Yes	Mult	F0	F2	F4	Int		No	Yes
Mult2									
Add									
Div									

1. Instruction Status :

Instr ⁿ . i j k	Issue	RO	EX	WB
LD F6, 34(R2)	1	2	3	4
LD F2, 45(R3)	5	6	7	
MUL F0, F2, F4	6			
SUB F8, F6, F2	7		I	
DIV F10, F0, F6				
ADD F6, F8, F2				

<http://www.flashbackrecorder.com/>
Watermark removed when fully licensed

3. Register Result Status :

	F0	F2	F4	F6	F8	F10	F12	F14
FU	Mult1	Int			Add			

2. Functional Unit Status :

FU	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer	Yes	Load	F2		R3				Yes
Mult1	Yes	Mult	F0	F2	F4	Int		No	Yes
Mult2									
Add	Yes	Sub	F8	F6	F2		Int	Yes	No
Div									

1. Instruction Status :

Instr ⁿ . i j k	Issue	RO	EX	WB
LD F6, 34(R2)	1	2	3	4
LD F2, 45(R3)	5	6	7	8
MUL F0, F2, F4	6			
SUB F8, F6, F2	7			
DIV F10, F0, F6	8			
ADD F6, F8, F2				

<http://www.flashbackrecorder.com/>
Watermark removed when fully licensed

3. Register Result Status :

	F0	F2	F4	F6	F8	F10	F12	F14
FU	Mult1				Add	Div		

2. Functional Unit Status :

FU	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer									
Mult1	Yes	Mult	F0	F2	F4			Yes	Yes
Mult2									
Add	Yes	Sub	F8	F6	F2			Yes	Yes
Div	Yes	Div	F10	F0	F6	Mult1		No	Yes

Cycle 9 to 11

1. Instruction Status :

Instr ⁿ . i j k	Issue	RO	EX	WB
LD F6, 34(R2)	1	2	3	4
LD F2, 45(R3)	5	6	7	8
MUL F0, F2, F4	6	9		
SUB F8, F6, F2	7	9	11	12
DIV F10, F0, F6	8			
ADD F6, F8, F2	I ₃			

3. Register Result Status :

	F0	F2	F4	F6	F8	F10	F12	F14
FU	Mult1					Div		

2. Functional Unit Status :

FU	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer									
Mult1	Yes	Mult	F0	F2	F4			Yes	Yes
Mult2									
Add									
Div	Yes	Div	F10	F0	F6	Mult1		No	Yes

1. Instruction Status :

Instr ⁿ . i j k	Issue	RO	EX	WB
LD F6, 34(R2)	1	2	3	4
LD F2, 45(R3)	5	6	7	8
MUL F0, F2, F4	6	9		
SUB F8, F6, F2	7	9	11	12
DIV F10, F0, F6	8			
ADD F6, F8, F2	I			

3. Register Result Status :

	F0	F2	F4	F6	F8	F10	F12	F14
FU	Mult1					Div		

2. Functional Unit Status :

FU	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer									
Mult1	Yes	Mult	F0	F2	F4			Yes	Yes
Mult2									
Add									
Div	Yes	Div	F10	F0	F6	Mult1		No	Yes

1. Instruction Status :

Instr ⁿ . i j k	Issue	RO	EX	WB
LD F6, 34(R2)	1	2	3	4
LD F2, 45(R3)	5	6	7	8
MUL F0, F2, F4	6	9		
SUB F8, F6, F2	7	9	11	12
DIV F10, F0, F6	8			
ADD F6, F8, F2	13			

I¹⁴

3. Register Result Status :

	F0	F2	F4	F6	F8	F10	F12	F14
FU	Mult1			Add		Div		

2. Functional Unit Status :

FU	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer									
Mult1	Yes	Mult	F0	F2	F4			Yes	Yes
Mult2									
Add	Yes	Add	F6	F8	F2			Yes	Yes
Div	Yes	Div	F10	F0	F6	Mult1		No	Yes

Cycle 14 to 16

<http://www.flashbackrecorder.com/>
Watermark removed when fully licensed

1. Instruction Status :

Instr ⁿ . i j k	Issue	RO	EX	WB
LD F6, 34(R2)	1	2	3	4
LD F2, 45(R3)	5	6	7	8
MUL F0, F2, F4	6	9	19	20
SUB F8, F6, F2	7	9	11	12
DIV F10, F0, F6	8			
ADD F6, F8, F2	13	14	16	

3. Register Result Status :

	F0	F2	F4	F6	F8	F10	F12	F14
FU				Add		Div		

2. Functional Unit Status :

FU	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer									
Mult1	Yes	Mult	F0	F2	F4			Yes	Yes
Mult2									
Add	Yes	Add	F6	F8	F2			Yes	Yes
Div	Yes	Div	F10	F0	F6	Mu		Yes	Yes

Cycle 16 to 22 – avoid WAR, WAW hazards

<http://www.flashbackrecorder.com/>
Watermark removed when fully licensed

1. Instruction Status :

Instr ⁿ . i j k	Issue	RO	EX	WB
LD F6, 34(R2)	1	2	3	4
LD F2, 45(R3)	5	6	7	8
MUL FO, F2, F4	6	9	19	20
SUB F8, F6, F2	7	9	11	12
DIV F10, F0, F6	8	21		
ADD F6, F8, F2	13	14	16	22

3. Register Result Status :

	F0	F2	F4	F6	F8	F10	F12	F14
FU						Div		

2. Functional Unit Status :

FU	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer									
Mult1	Yes								
Mult2									
Add									
Div	Yes	Div	F10	F0	F6			Yes	Yes

Cycle 22 to 62

1. Instruction Status :

<http://www.flashbackrecorder.com/>
Watermark removed when fully licensed

Instr ⁿ . i j k	Issue	RO	EX	WB
LD F6, 34(R2)	1	2	3	4
LD F2, 45(R3)	5	6	7	8
MUL F0, F2, F4	6	9	19	20
SUB F8, F6, F2	7	9	11	12
DIV F10, F0, F6	8	21	61	62
ADD F6, F8, F2	13	14	16	22

3. Register Result Status :

	F0	F2	F4	F6	F8	F10	F12	F14
FU								

I

2. Functional Unit Status :

FU	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer									
Mult1									
Mult2									
Add									
Div									

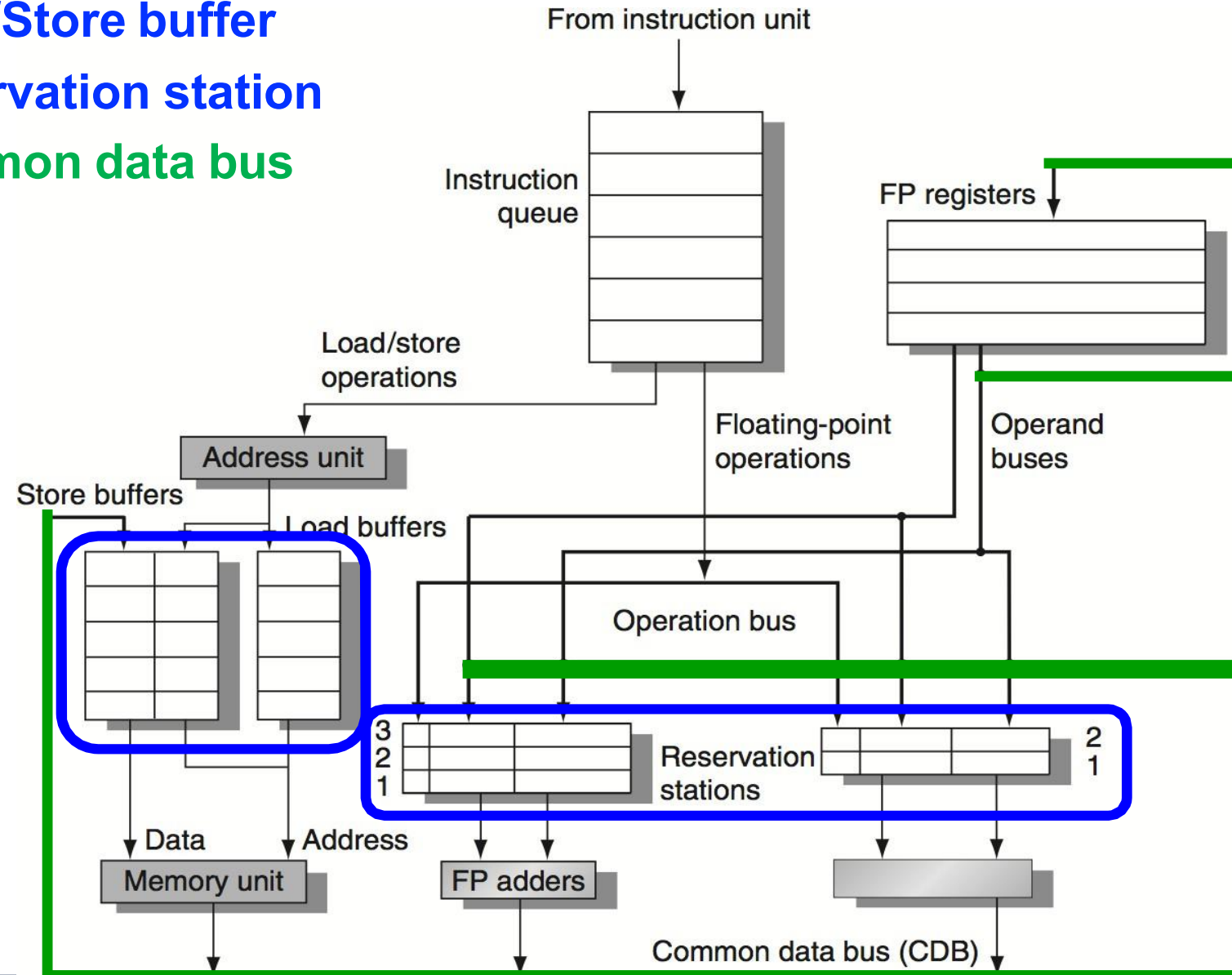
Tomasulo Example

Reference:

<https://www.youtube.com/watch?v=EfwCfhXxNnI&list=WL&index=6>

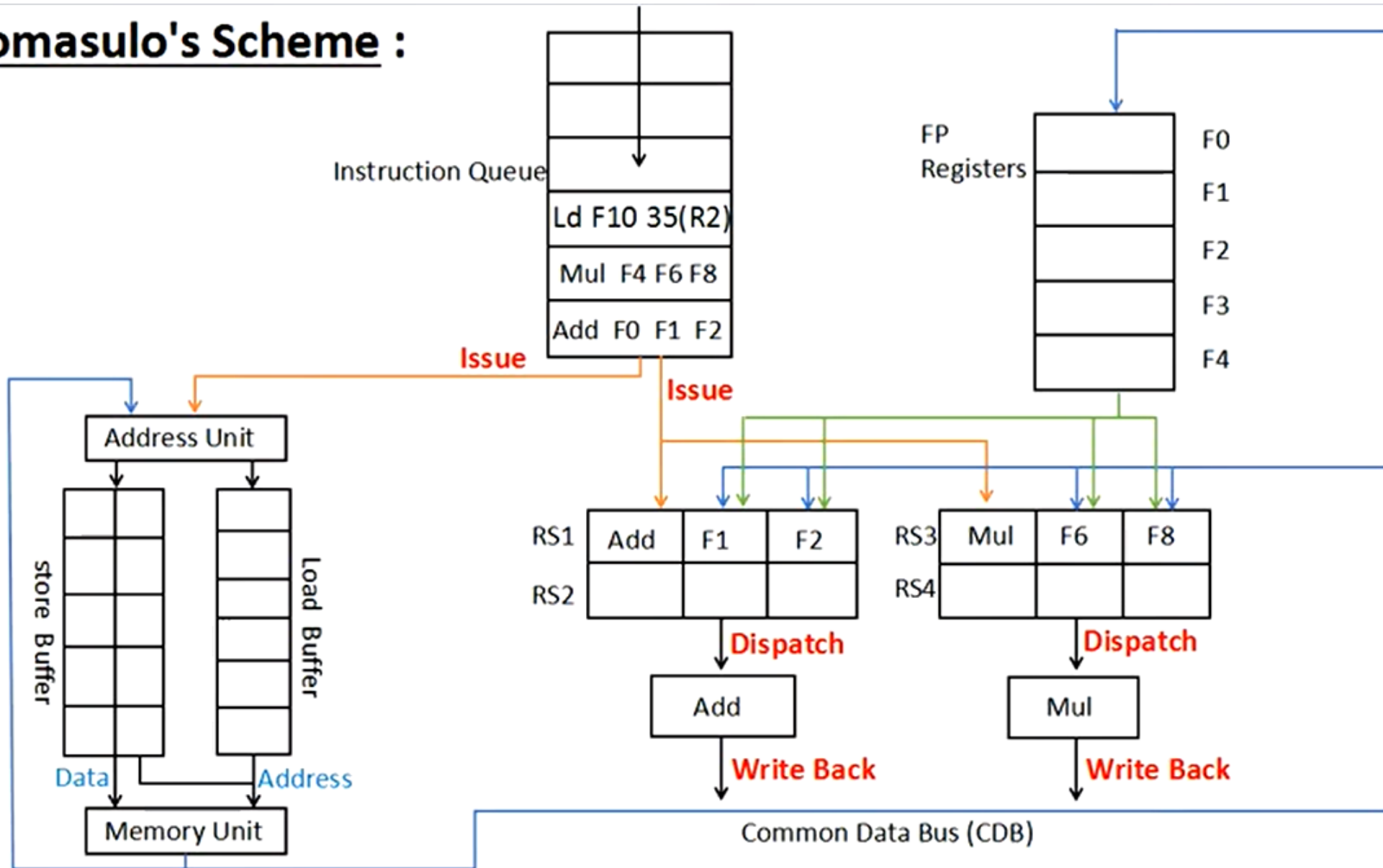
Organizations of Tomasulo's Algorithm in RISC-V

- Load/Store buffer
- Reservation station
- Common data bus



Simplified Tomasulo

Tomasulo's Scheme :



3 Parts of Tomasulo



3 Parts Of Tomasulo's Scheme :

1. Instruction Status : In which of the 3 stages(Issue,EX,WB) the instruction is.

2.Reservation Stations Status :

- i. **Busy** : Whether the reservation unit and its accompanying functional unit is busy or not.
- ii. **Op** : Operation to be performed.
- iii. **Q_j , Q_k** :Functional Units producing source operands.
- iv. **V_j , V_k** : Value of the source operands.

3. Register Result Status :Which functional unit is going to write the register.

Question on Tomasulo

Q. Write the steps of execution using Tomasulo's Algorithm. Assume we have 3 loader, 3 adders, 2 multiplier. The following set of MIPS instruction is going to be executed in a pipelined system.

- Reference:

- <https://www.youtube.com/watch?v=EfwCfhXxNnI&list=WL&index=6>

LD F6, 34(R2)
LD F2, 45(R3)
MUL F0, F2, F4
SUB F8, F6, F2
DIV F10, F0, F6
ADD F6, F8, F2

The latencies of - Load instruction is 2 cycle
Add instruction is 2 cycle
Multiply instruction is 10 cycle
Divide instruction is 40 cycles

Important Parts of Tomasulo – for Solution

Instruction Status

Instr	Issue	EX	WB
LD F6, 34(R2)			
LD F2, 45(R3)			
MUL F0, F2, F4			
SUB F8, F6, F2			
DIV F10, F0, F6			
ADD F6, F8, F2			

The latencies of - Load instruction is 2 cycle
 Add instruction is 2 cycle
 Multiply instruction is 10 cycle
 Divide instruction is 40 cycles

<http://www.flashbackrecorder.com/>

Watermark removed when fully licensed

	Busy	Address
Load1		
Load2		
Load3		

Reservation Station Status

Name	Busy	Op	V_j	V_k	Q_j	Q_k
Add1						
Add2						
Add3						
Mult1						
Mult2						

Register Result Status

	F0	F2	F4	F6	F8	F10		F30
FU								

Cycle 1

Instruction Status

Instr	Issue	EX	WB
LD F6, 34(R2)	1	I	
LD F2, 45(R3)			
MUL F0, F2, F4			
SUB F8, F6, F2			
DIV F10, F0, F6			
ADD F6, F8, F2			

The latencies of - Load instruction is 2 cycle
 Add instruction is 2 cycle
 Multiply instruction is 10 cycle
 Divide instruction is 40 cycles

<http://www.flashbackrecorder.com/>
 Watermark removed when fully licensed

	Busy	Address
Load1	yes	34+R2
Load2		
Load3		

Reservation Station Status

Name	Busy	Op	V_j	V_k	Q_j	Q_k
Add1						
Add2						
Add3						
Mult1						
Mult2						

Register Result Status

	F0	F2	F4	F6	F8	F10		F30
FU				Load1				

Cycle 2

Instruction Status

Instr	Issue	EX	WB
LD F6, 34(R2)	1	2-	
LD F2, 45(R3)	2		
MUL F0, F2, F4		I	
SUB F8, F6, F2			
DIV F10, F0, F6			
ADD F6, F8, F2			

<http://www.flashbackrecorder.com/>
Watermark removed when fully licensed

	Busy	Address
Load1	yes	34+R2
Load2	Yes	45+R3
Load3		

Reservation Station Status

Name	Busy	Op	V_j	V_k	Q_j	Q_k
Add1						
Add2						
Add3						
Mult1						
Mult2						

Register Result Status

	F0	F2	F4	F6	F8	F10		F30
FU		Load2		Load1				

Cycle 3

Instruction Status

Instr	Issue	EX	WB
LD F6, 34(R2)	1	2-3	
LD F2, 45(R3)	2	3-	
MUL F0, F2, F4	3		
SUB F8, F6, F2			
DIV F10, F0, F6			
ADD F6, F8, F2			

<http://www.flashbackrecorder.com/>
Watermark removed when fully licensed

	Busy	Address
Load1	yes	34+R2
Load2	Yes	45+R3
Load3		

Reservation Station Status

Name	Busy	Op	V _j	V _k	Q _j	Q _k
Add1						
Add2						
Add3						
Mult1	Yes	Mult		R(F4)	Load2	
Mult2			I			

Register Result Status

	F0	F2	F4	F6	F8	F10		F30
FU	Mult1	Load2		Load1				

Cycle 4

Instruction Status

Instr	Issue	EX	WB
LD F6, 34(R2)	1	2-3	4
LD F2, 45(R3)	2	3-4	I
MUL F0, F2, F4	3		
SUB F8, F6, F2	4		
DIV F10, F0, F6			
ADD F6, F8, F2			

<http://www.hasnabackrecorder.com/>
Watermark removed when fully licensed

	Busy	Address
Load1		
Load2	Yes	45+R3
Load3		

Reservation Station Status

Name	Busy	Op	V _j	V _k	Q _j	Q _k
Add1	Yes	Sub	M(A1)			Load2
Add2						
Add3						
Mult1	Yes	Mult		R(F4)	Load2	
Mult2						

Register Result Status

	F0	F2	F4	F6	F8	F10		F30
FU	Mult1	Load2		M(A1)	Add1			

Cycle 5

Instruction Status

Instr	Issue	EX	WB
LD F6, 34(R2)	1	2-3	4
LD F2, 45(R3)	2	3-4	5
MUL F0, F2, F4	3		
SUB F8, F6, F2	4		
DIV F10, F0, F6	5		
ADD F6, F8, F2	I		

<http://www.flashbackrecorder.com/>
Watermark removed when fully licensed

	Busy	Address
Load1		
Load2		
Load3		

Reservation Station Status

Name	Busy	Op	V_j	V_k	Q_j	Q_k
Add1	Yes	Sub	M(A1)	M(A2)		
Add2						
Add3						
Mult1	Yes	Mult	M(A2)	R(F4)		
Mult2	Yes	Div		M(A2)	Add1	

Register Result Status

	F0	F2	F4	F6	F8	F10		F30
FU	Mult1	M(A2)		M(A1)	Add1	Mult2		

Cycle 6

Instruction Status

Instr	Issue	EX	WB
LD F6, 34(R2)	1	2-3	4
LD F2, 45(R3)	2	3-4	5
MUL F0, F2, F4	3	6-	
SUB F8, F6, F2	4	6-7	8
DIV F10, F0, F6	5		
ADD F6, F8, F2	6		

http://www.flashbackrecorder.com
Watermark removed when fully licensed

	Busy	Address
Load1		
Load2		
Load3		

Reservation Station Status

Name	Busy	Op	V _j	V _k	Q _j	Q _k
Add1						
Add2	Yes	Add		M(A2)	Add1	
Add3						
Mult1	Yes	Mult	M(A2)	R(F4)		
Mult2	Yes	Div		M(A2)	Add1	

Register Result Status

	F0	F2	F4	F6	F8	F10		F30
FU	Mult1	M(A2)		Add2	M1-M2	Mult2		

Cycle 7 to 11

Instruction Status

Instr	Issue	EX	WB
LD F6, 34(R2)	1	2-3	4
LD F2, 45(R3)	2	3-4	5
MUL F0, F2, F4	3	6-	
SUB F8, F6, F2	4	6-7	8
DIV F10, F0, F6	5	I	
ADD F6, F8, F2	6	9-10	11

	Busy	Address
Load1		
Load2		
Load3		

Reservation Station Status

Name	Busy	Op	V_j	V_k	Q_j	Q_k
Add1						
Add2						
Add3						
Mult1	Yes	Mult	M(A2)	R(F4)		
Mult2	Yes	Div		M(A2)	Mult1	

Register Result Status

	F0	F2	F4	F6	F8	F10		F30
FU	Mult1	M(A2)		M1-M2+M(A2)	M1-M2	Mult2		

Cycle 12 to 16

Instruction Status

Instr	Issue	EX	WB
LD F6, 34(R2)	1	2-3	4
LD F2, 45(R3)	2	3-4	5
MUL FO, F2, F4	3	6-15	16
SUB F8, F6, F2	4	6-7	8
DIV F10, F0, F6	5	I	
ADD F6, F8, F2	6	9-10	11

	Busy	Address
Load1		
Load2		
Load3		

Reservation Station Status

Name	Busy	Op	V_j	V_k	Q_j	Q_k
Add1						
Add2						
Add3						
Mult1						
Mult2	Yes	Div	$M(A2)*R$	$M(A2)$		

Register Result Status

	F0	F2	F4	F6	F8	F10		F30
FU	$M(A2)*R$	$M(A2)$		$M1-M2+M(A2)$	$M1-M2$	Mult2		

Cycle 12 to 57

Instruction Status

Instr	Issue	EX	WB
LD F6, 34(R2)	1	2-3	4
LD F2, 45(R3)	2	3-4	5
MUL F0, F2, F4	3	6-15	16
SUB F8, F6, F2	4	6-7	8
DIV F10, F0, F6	5	17- 56	57
ADD F6, F8, F2	6	9-10	11

<http://www.flashbackrecorder.com/>

Watermark removed when fully licensed

	Busy	Address
Load1		
Load2		
Load3		

Reservation Station Status

Name	Busy	Op	V_j	V_k	Q_j	Q_k
Add1						
Add2						
Add3						
Mult1						
Mult2						

Register Result Status

	F0	F2	F4	F6	F8	F10		F30
FU	$M(A2)*R$	$M(A2)$		$M1-M2+M(A2)$	$M1-M2$	T		

Reading Notes

Readings:
Chapter 3 of course textbook

Also watch youtube examples of Scoreboard and Tomasulo as mentioned later

High Performance Computer Architecture - Scoreboarding Example



HARSHIT ANAND



High Performance Computer Architecture - Tomasulo's Algorithm Example



HARSHIT ANAND

